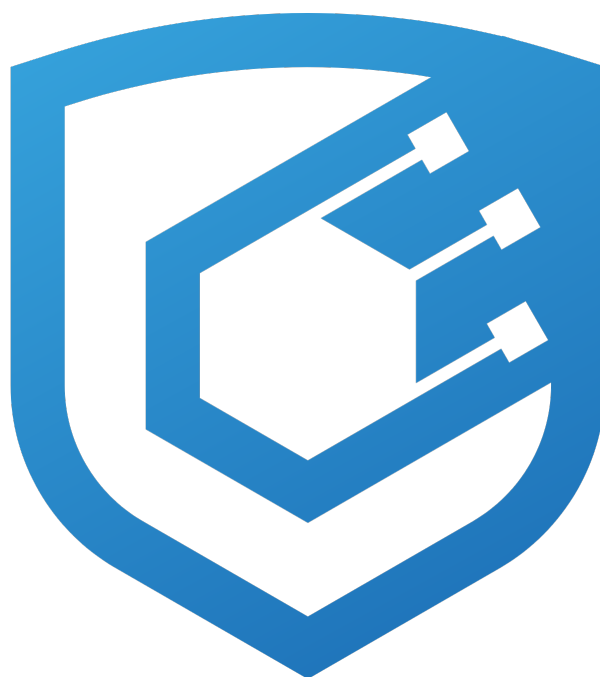




Highway Substrate Audit Report

Prepared by [Cyfrin](#)

Version 2.1

Lead Auditors

[Farouk](#)

[qpzm](#)

September 29, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
4.1	Module / Component Overview	3
4.2	Message Lifecycle	4
4.3	Trust Model and Roles	4
4.4	Cross-Chain Consistency Model (the load-bearing invariant)	5
4.5	BLS12-381 Committee Attestation	5
4.6	Key Design Properties	5
5	Audit Scope	5
6	Executive Summary	6
7	Findings	11
7.1	Critical Risk	11
7.1.1	HighwayEntry::verify_bls_attestation counts all 128 relayer_bitmap bits but aggregates only the low committee_size seats	11
7.2	Medium Risk	15
7.2.1	NftDelegationRegistry::delegate Path A never binds the operator NFT caller to the relayer's manager	15
7.2.2	DefaultUndelegationCooldown=0 collapses the slash window between undelegate and complete_undelegation	15
7.2.3	HighwayEntry::execute_inbound_asset_operation MINT to a provider-less fresh recipient can fail the pallet_assets minimum-balance check	16
7.2.4	Inbound payload dispatches with the delivering relayer's Signed origin; a whitelisted Balances transfer drains the relayer	17
7.2.5	Caller-chosen relayer ID and slot amplify a sub-third coalition's committee-takeover probability by over 9,500x	20
7.3	Low Risk	25
7.3.1	Pallet::do_update_active_set and verify_bls_attestation never read the is_authorized_operator gate	25
7.3.2	Pallet::remove_relayer guards only current and pending epochs while the verify_bls_attestation pin window keeps the current-1 committee claimable	25
7.3.3	HighwayConfig::register_token stores an admin-supplied local_decimals never reconciled against the asset's pallet_assets metadata decimals	26
7.3.4	HighwayEntry::execute_inbound_native_operation discards the deposit_creating imbalance on a native MINT	27
7.3.5	NftDelegationRegistry::complete_undelegation is permissionless and lets a bystander force-complete a pending undelegation	28
7.3.6	Deployed CommitteeSize=34 and MinSignatures=24 violate the pallet integrity_test floor MinSignatures>=87	28
7.3.7	NftDelegationRegistry::slash_delegation lowers RelayerDelegatedWeight but never the RelayerOperatorTier capacity ceiling	29
7.3.8	NftDelegationRegistry::complete_undelegation Path A undelegation orphans the Path B delegators	30
7.3.9	HighwayEntry::execute_inbound_native_operation uses AllowDeath on the native RELEASE from a vault shared with asset corridors	30
7.3.10	HighwayConfig::configure_native_token_for_chain accepts a min_amount below the existential deposit when native_decimals <= source_decimals	31

7.3.11	Pallet:: <code>select_committee</code> orders <code>active_relayers</code> ascending by id and depends on the off-chain builder agreeing on that ordering	31
7.3.12	Bootstrap scripts register only 16 relayers while the deployed runtime requires <code>Committee-Size=34</code>	32
7.3.13	BLS committee private keys and libp2p private keys are committed to the repository	33
7.3.14	<code>HighwayEntry::verify_bls_attestation</code> enforces no upper bound on the relayer-supplied <code>ttl</code>	33
7.3.15	<code>Pallet::receive_message</code> folds <code>payload_target_address</code> into the <code>messageId</code> but discards it during execution	34
7.3.16	<code>NftDelegationRegistry::clear_relayer_delegations</code> removes <code>RelayerOperatorTier</code> but never calls <code>deauthorize_operator</code>	34
7.3.17	A relayer operator's own Path-A NFT weight counts against the relayer's delegation capacity ceiling	35
7.3.18	<code>highway_config</code> implements a single mutable <code>Admin</code> while <code>scripts/README.md</code> asserts an immutable <code>max-2-admin</code> set	35
7.3.19	<code>HighwayEntry::receive_message</code> weight is benchmarked over a <code>Linear<87,128></code> signer domain that diverges from the deployed 34-member committee	36
7.3.20	<code>receive_message</code> declared weight omits the dispatched payload <code>RuntimeCall</code> , breaking the weight upper-bound invariant	37
7.3.21	Payload whitelist keys on raw <code>payload[0..2]</code> instead of the decoded <code>RuntimeCall</code> identity	39
7.3.22	<code>RelayerRegistry::update_bls_key</code> overwrites a seated relayer's BLS key with no pin-window coordination, invalidating in-flight attested proofs	41
7.3.23	<code>complete_undelegation</code> charges the flat worst-case weight and shifts with $O(n)$ <code>remove</code>	42
7.3.24	Proof-of-Possession reuses the message-signing ciphersuite/DST instead of a dedicated <code>_POP_DST</code>	43
7.3.25	<code>clear_relayer_delegations</code> fail-fasts on one un-unbindable NFT, bricking admin recovery	44
7.3.26	A public Proof-of-Possession can be replayed to re-seat a freed BLS key onto a different relayer	45
7.3.27	Permissionless <code>pallet_assets</code> creation plus no asset-control binding in <code>register_token</code> lets a third party own a bridged asset	46
7.3.28	<code>pallet_nfts</code> bound type <code>WeightInfo = ()</code> with permissionless <code>CreateOrigin</code> enables zero-weight block stuffing by any account	47
7.3.29	<code>remove_chain</code> becomes permanently un-callable once <code>TokenChainConfig</code> exceeds <code>RemoveChainMaxScan</code>	48
7.3.30	The <code>highway-nft-permission</code> soft-lock model has four related lock-integrity and stale-ownership gaps across <code>bind / unbind</code> and the delegation registry	49
7.3.31	<code>clear_relayer_delegations</code> declares a fixed max-size proof above the runtime limits, making the normal admin path inadmissible	52
7.3.32	<code>emit_message</code> accepts an over-width EVM token target and burns funds for an unclaimable message	55
7.4	Informational	58
7.4.1	<code>dev_chain_spec.json</code> genesis is stale versus the audited runtime and encodes a plural <code>highwayConfig.admins</code> array	58

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

Unless stated otherwise, runtime, deployment, and configuration values used in Cyfrin's analysis are those found in the reviewed code and materials at the audited commit. Later remediation values and Highway-provided operational values are identified as such. These values should not be read as those used by any production deployment, and Cyfrin did not independently verify any production deployment's configuration. This qualification does not affect conclusions based on a value identified in the report or a Highway response as fixed in the relevant code revision.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Highway (the Mosaic Bridge) is a cross-chain token-and-message bridge. This summary covers its Substrate implementation, the audit scope; the bridge also operates EVM (Ethereum) and Solana legs that act as this runtime's cross-chain counterparties. A message emitted on a source chain is attested by a BLS12-381 committee drawn from a shared relayer set, and executed on the destination chain, where it mints or releases tokens and/or dispatches a whitelisted payload call. A defining property of the system is that the message identity and the committee-attestation format must be reproduced byte-for-byte by each leg's independent implementation, so these pallets must agree exactly with their EVM and Solana counterparties on those encodings.

Production usage: every cross-chain transfer or message touches the full lifecycle. A user calls the source chain's emit entrypoint, which debits funds (burn or escrow), assigns a `messageId`, and emits the message. Off-chain relayers in the active committee observe it and produce a BLS aggregate attestation over a canonical payload. Any one committee relayer submits the attestation to the destination chain's `receive_message` extrinsic, which re-derives the `messageId`, verifies the aggregate against the committee it independently selects for that slot, marks the message processed (replay protection), and then mints/releases the tokens and dispatches the optional payload.

4.1 Module / Component Overview

Pallet	Role
highway-entry	Inbound/outbound entrypoints: <code>emit_message</code> (burn/escrow, <code>messageId</code> assignment) and <code>receive_message</code> (BLS aggregate verification, deterministic committee selection, mint/release, payload dispatch via <code>execute_payload</code>)
highway-registry	Relayer registration, BLS key + proof-of-possession, operational keys, registered-relayer bitmap, active-set epoch ring buffer, <code>is_authorized_operator</code> flag
highway-config	Admin surface, token & corridor configuration, decimal handling, (<code>pallet_index</code> , <code>call_index</code>) payload whitelist
highway-nft-delegation-registry	Relayer backing and delegation: Path A operator self-bind and Path B external delegation, tier ceilings, slashing, undelegation lifecycle
highway-nft-types / highway-nft-permission	NFT tier definitions (Nomad / Anchor / Vanguard capacity tiers) and permission-token minting for PoA governance
highway-bls-utils	BLS12-381 primitives, EIP-2537 byte-format helpers, hash-to-curve support

4.2 Message Lifecycle

1. **Outbound (source).** The user calls `emit_message` with a target chain, optional token leg (`tokenId`, target address, amount) and optional payload leg (target address, encoded call), plus fee handling. The source debits the funds (Burn or Escrow mode), allocates a nonce, computes `messageId = H(domain ++ sourceChainId ++ targetChainId ++ ... ++ nonce ++ optional fields)`, and emits the message.
2. **Attestation (off-chain committee).** The relayers seated in the active committee for the message's slot co-sign a canonical BLS payload (`messageId ++ ttl ++ slot ++ relayerId`) and produce a single aggregate signature plus a committee bitmap.
3. **Inbound (destination).** A committee relayer submits the attestation via `receive_message`. The destination: checks replay against `ExecutedInboundMessages`, re-derives `messageId` and asserts equality, selects the committee deterministically from its own active set and epoch randomness, aggregates the bitmap-selected registered public keys on-chain (reading each `relayer_bls_key` live), verifies the aggregate against a quorum threshold, marks the message processed, then performs the token leg (mint or release) and the payload leg (`execute_payload` dispatches the whitelisted call).

4.3 Trust Model and Roles

- **Relayer committee (the security root).** Inbound execution trusts a threshold BLS aggregate from the active committee. A quorum must co-sign; the audited configuration was `CommitteeSize = 34` with `MinSignatures = 24`, which the team has since raised to `CommitteeSize = 128` with `MinSignatures = 86`. Committee membership is selected deterministically per slot via keccak-based rejection sampling over the active set and epoch randomness, so the destination reconstructs exactly who must have signed.
- **Active set and epochs.** The active relayer set is rotated by epoch through an admin / authorized-updater path (`do_update_active_set`). Retained per-epoch bitmaps let a proof claim against the current or current - 1 epoch (a pin window), and a `MinValidEpoch` floor provides epoch-wide revocation. Each relayer registers a BLS key with a proof-of-possession before it can be seated.
- **Delegation and backing (Substrate-specific).** Relayer capacity is gated by admin-minted NFTs: an operator self-binds a relayer-tier NFT (Path A), which sets the relayer's `max_delegation_capacity` ceiling by

tier (Nomad 140k / Anchor 300k / Vanguard 1M), and external parties delegate up to that ceiling (Path B). `RelayerDelegatedWeight` tracks the running total.

- **Admin roles.** A single mutable `Admin StorageValue` plus an `AuthorizedRegistrars` set gate the privileged config and registry mutations; there is no on-chain multi-admin bound.

4.4 Cross-Chain Consistency Model (the load-bearing invariant)

Because a message is identified by a `messageId` that each chain recomputes independently, and because the committee attests a canonical payload that each chain independently reconstructs, the bridge is correct only if all three legs agree on the application-level `messageId`, attestation preimage, and committee-selection rules. A divergence can fail closed and strand a message or, where it does not fail closed, create replay or substitution risk. At the audited commit, this leg used `HWY_MSG_V1` with SCALE encoding and had adopted the attestation epoch and `MinValidEpoch` revocation floor. Current Substrate production source has since adopted the network-bound canonical V1 message-ID layout and the 94-byte epoch-bound `HWY_BLS_V1` attestation preimage reflected in current EVM source. EVM PR 97 closed without merge; PR 129 is its operative ciphersuite replacement, and the encoding changes from EVM PR 126 are present in current Substrate source. Solana shares the V1 application preimages but uses curve-specific BN254 verification. The checked-in Substrate helper and diagnostic fixture remain on the pre-epoch path and therefore do not demonstrate end-to-end release parity.

4.5 BLS12-381 Committee Attestation

Public keys are in G1 and signatures in G2 (`min_pk`); messages are hashed to G2 through RFC 9380 SSWU_RO. Rogue-key resistance relies on proof of possession at registration. At the audited commit, the proof-of-possession path reused the committee-signing `_NUL_` ciphersuite. Current production source retains `_NUL_` for committee attestations and uses the dedicated `_POP_` ciphersuite for proof of possession. Unlike the EVM leg, which implements the curve over the EIP-2537 precompiles, this runtime relies on the platform's BLS primitives; the on-chain verifier reconstructs the committee aggregate from each seated relayer's registered key. Proof-of-possession key serialization remains leg-specific.

4.6 Key Design Properties

- **Value conservation per corridor.** Each (token, chain) corridor is configured as `Burn/Mint` or `Escrow/Release`; `validate_bridge_configuration` keeps lock and unlock paired and requires an escrow account exactly when the corridor needs one.
- **Replay protection.** Each inbound `messageId` is recorded in `ExecutedInboundMessages` before value commits, so a given message executes at most once per destination and a reverted delivery remains re-deliverable.
- **On-chain trustless aggregation.** The destination aggregates the committee public keys itself rather than accepting a submitted aggregate, checking each supplied key against the stored registry.
- **Deterministic committee selection.** Committee seats are derived from epoch randomness and slot via rejection sampling, reproduced identically across legs so the attesting set and the verifying set coincide.
- **Epoch pinning.** Retained per-epoch bitmaps plus the `current / current - 1` window let an in-flight proof survive one rotation without re-attestation, with `MinValidEpoch` as an epoch-wide revocation floor.
- **Fail-closed inbound.** Verification failures (wrong `messageId`, insufficient signers, unregistered key) revert; a divergent encoding stalls delivery rather than mis-delivering, and inbound execution is idempotent-guarded before value moves.

5 Audit Scope

The audit scope was limited to:

```
pallets/highway-bls-utils/src/lib.rs
pallets/highway-config/src/lib.rs
pallets/highway-entry/src/lib.rs
pallets/highway-nft-delegation-registry/src/lib.rs
pallets/highway-nft-permission/src/lib.rs
pallets/highway-nft-types/src/lib.rs
pallets/highway-registry/src/lib.rs
```

6 Executive Summary

Over the course of 15 days, the Cyfrin team conducted an audit on the [Highway Substrate](#) code provided by [Highway](#). The findings consist of 1 Critical, 5 Medium & 32 Low severity issues with the remainder being informational.

Cyfrin reviewed the Substrate implementation of Highway (the Mosaic Bridge), a cross-chain token-and-message bridge secured by a BLS12-381 relayer committee. The review covered the FRAME pallets: the inbound and outbound entypoints (`highway-entry`), the relayer registry and active set (`highway-registry`), the admin, corridor, and payload-whitelist surface (`highway-config`), the NFT-gated delegation and slashing stack (`highway-nft-delegation-registry`, `highway-nft-types`, `highway-nft-permission`), and the BLS utilities (`highway-bls-utils`). At risk are the funds escrowed or mint-controlled by these pallets and the integrity of the inbound messages that move them. This leg is one of three (EVM, Substrate, Solana) that must reproduce the same message identity and attestation format byte-for-byte.

A message is identified by a `messageId` and authenticated by a committee attestation that each chain recomputes independently, so these pallets are correct only when their encodings stay consistent with the rest of the system. We focused on where that agreement can break or where inbound execution can be steered away from what the committee attested: the aggregate-verification and committee-selection path; the `messageId` and BLS attestation-payload encoding (domain separation, ciphersuite, proof-of-possession); the inbound path that mints, releases, or dispatches on caller-supplied parameters; the relayer registry and epoch lifecycle; the NFT-gated delegation and slashing accounting; and the access-control and deployment surface that arms all of it.

The most severe finding is in the aggregate verifier itself. `verify_bls_attestation` counts signers across the full 128-bit committee bitmap (`count_ones()`) but aggregates public keys only over `0..committee_size`, so a submitter can set one real in-committee seat plus padding bits in `[committee_size, 128)` to clear both the `MinSignatures` and `committee_size` bounds while only a single key enters the BLS aggregate. With the audited `CommitteeSize = 34`, a single colluding or compromised committee relayer forges a threshold-passing attestation and gains unilateral inbound mint/release, a full bridge compromise; we classify this as Critical. The team has since raised the runtime configuration to `CommitteeSize = 128` with `MinSignatures = 86` and added an explicit guard that rejects bitmap bits outside `[0, committee_size)` before the threshold check. The integrity test also requires a supported committee width of 128. Together, these changes close the audited exploit and the counted-set versus aggregated-set mismatch: every bit counted for an accepted attestation corresponds to a committee seat whose public key enters BLS verification. A future sub-128 configuration would still reject out-of-window bits and therefore would not recreate the threshold bypass. The second cluster of inbound risk is payload dispatch: `execute_payload` runs the decoded `RuntimeCall` under `RawOrigin::Signed(relayer)`, so a whitelisted funds-moving call (for example `Balances::transfer_all`) executes with the delivering relayer's own account authority and drains it, a borrowed-authority confused deputy. The intended neutral, keyless origin (`pallet_account_id`) is specced but left as dead code. The realized impact is gated by the admin-controlled whitelist, which ships as `System::remark` only, so we grade it Medium under a trusted-operator model rather than a permissionless drain; it becomes a full drain the moment a token-transfer call is whitelisted, which for a bridge is the expected production configuration.

The encoding-consistency cluster mirrors the EVM leg. The audited proof-of-possession path reused the message-signing DST, and its preimage differed from the other legs. Current Substrate production source uses the network-bound `HWY_MSG_V1` message-ID format, an epoch-bound 94-byte `HWY_BLS_V1` committee-attestation preimage with the `_NUL_` signing ciphersuite, and a dedicated `_POP_` proof-of-possession ciphersuite; the proof-of-possession key serialization remains leg-specific. EVM PR 97 closed without merge; PR 129 is its operative replacement, while the consolidated encoding changes from EVM PR 126 are reflected in current Substrate production source. Solana shares the V1 application preimages but uses curve-specific BN254 verification. The checked-in Substrate signing

and receive helper and diagnostic compatibility fixture remain on the pre-epoch path, so they do not establish a current successful end-to-end flow.

The remaining findings fall into five areas. **(1) Committee, threshold, and relayer lifecycle:** the active-set bitmap is not checked against the `is_authorized_operator` flag (the corresponding error is dead code), the audited committee parameters sat below the benchmarked signer range (resolved by the same raise to 128 / 86), the default active-set update path auto-revokes prior epochs, and key mutators (`update_bls_key`, `remove_relayer`) are uncoordinated with the pin window, so rotating or removing a seated relayer's key invalidates in-flight attested proofs (bounded liveness, re-attest next epoch). **(2) BLS input hygiene:** the registration path leaves gaps around point-at-infinity, on-curve, and subgroup validation and PoP pubkey consistency. **(3) Value-movement coupling and weight accounting:** `receive_message` declares a weight that omits the dispatched payload call (a weight upper-bound violation, bounded by the 4 KiB payload cap and the remark-only whitelist so small as deployed), the payload whitelist keys on the raw payload[0..2] prefix rather than the decoded call identity (latent, opens under a runtime-upgrade reindex), and native mint/release has existential-deposit edge cases (a silent sub-ED `deposit_creating`, `AllowDeath` on a shared escrow). **(4) Delegation, slashing, and NFTs (Substrate-specific):** slashing decrements delegated weight but never lowers the tier ceiling or deauthorizes, so governance punishment inverts and reopens the full 2x capacity against a zero-backing relayer; the undelegation cooldown defaults to zero; Path A never binds the caller to the relayer's operator; and teardown paths orphan Path B delegators or omit `deauthorize_operator`. **(5) Access control, bootstrap, and configuration:** a single mutable admin against a documented bounded set, genesis placeholders, permissionless `pallet_assets` creation front-running, and deployment-tooling mismatches. A separate arithmetic review of the pallets found no unsafe numeric cast on any security- or fund-relevant path (the amount-conversion path uses `checked_pow + try_into`), so that class is clean.

Several items are acknowledged or deferred consistently with their EVM disposition: immediate BLS-key invalidation on `update_bls_key` is intended behavior for compromised-key rotation and was ruled won't-change on the EVM leg, so the actionable item there is documentation plus an optional lossless-rotation path. The network-bound message-ID, epoch-bound attestation preimage, and proof-of-possession ciphersuite changes are present in current production source; release-level interoperability still depends on coordinated end-to-end validation.

Beyond the resolved committee-bitmap bypass, the review's recommendations include: dispatch inbound payloads from the keyless `pallet_account_id` origin and keep that account fund-less and role-less, and gate the whitelist on the decoded `RuntimeCall` identity rather than the raw prefix; harden BLS registration with on-curve, subgroup, and non-infinity checks; make slashing and delegation state transitions consistent (clear the tier ceiling and deauthorize on zero backing, set a nonzero cooldown, bind Path A to the operator); and maintain cross-leg format compatibility through versioned vectors, coordinated releases, and end-to-end tests.

Summary

Project Name	Highway Substrate
Repository	hway-substrate
Commit	df99567ea908...
Audit Timeline	June 29th - July 17th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Partially Resolved	Acknowledged
Critical Risk	1	1	0	0
High Risk	0	0	0	0
Medium Risk	5	3	0	2
Low Risk	32	20	1	11
Informational	1	1	0	0
Gas Optimizations	0	0	0	0
Total	39	25	1	13

Summary of Findings

[C-1] HighwayEntry::verify_bls_attestation counts all 128 relayer_bitmap bits but aggregates only the low committee_size seats	Resolved
[M-1] NftDelegationRegistry::delegate Path A never binds the operator NFT caller to the relayer's manager	Resolved
[M-2] DefaultUndelegationCooldown=0 collapses the slash window between undelegate and complete_undelegation	Acknowledged
[M-3] HighwayEntry::execute_inbound_asset_operation MINT to a provider-less fresh recipient can fail the pallet_assets minimum-balance check	Resolved
[M-4] Inbound payload dispatches with the delivering relayer's Signed origin; a whitelisted Balances transfer drains the relayer	Resolved
[M-5] Caller-chosen relayer ID and slot amplify a sub-third coalition's committee-takeover probability by over 9,500x	Acknowledged
[L-01] Pallet::do_update_active_set and verify_bls_attestation never read the is_authorized_operator gate	Resolved
[L-02] Pallet::remove_relayer guards only current and pending epochs while the verify_bls_attestation pin window keeps the current-1 committee claimable	Resolved
[L-03] HighwayConfig::register_token stores an admin-supplied local_decimals never reconciled against the asset's pallet_assets metadata decimals	Resolved
[L-04] HighwayEntry::execute_inbound_native_operation discards the deposit_creating imbalance on a native MINT	Resolved
[L-05] NftDelegationRegistry::complete_undelegation is permissionless and lets a bystander force-complete a pending undelegation	Acknowledged
[L-06] Deployed CommitteeSize=34 and MinSignatures=24 violate the pallet integrity_test floor MinSignatures>=87	Resolved
[L-07] NftDelegationRegistry::slash_delegation lowers RelayerDelegatedWeight but never the RelayerOperatorTier capacity ceiling	Acknowledged

[L-08] NftDelegationRegistry::complete_undelegation Path A undelegation orphans the Path B delegators	Acknowledged
[L-09] HighwayEntry::execute_inbound_native_operation uses AllowDeath on the native RELEASE from a vault shared with asset corridors	Resolved
[L-10] HighwayConfig::configure_native_token_for_chain accepts a min_amount below the existential deposit when native_decimals <= source_decimals	Resolved
[L-11] Pallet::select_committee orders active_relayers ascending by id and depends on the off-chain builder agreeing on that ordering	Acknowledged
[L-12] Bootstrap scripts register only 16 relayers while the deployed runtime requires CommitteeSize=34	Resolved
[L-13] BLS committee private keys and libp2p private keys are committed to the repository	Resolved
[L-14] HighwayEntry::verify_bls_attestation enforces no upper bound on the relayer-supplied ttl	Acknowledged
[L-15] Pallet::receive_message folds payload_target_address into the messageId but discards it during execution	Acknowledged
[L-16] NftDelegationRegistry::clear_relayer_delegations removes RelayerOperatorTier but never calls deauthorize_operator	Resolved
[L-17] A relayer operator's own Path-A NFT weight counts against the relayer's delegation capacity ceiling	Resolved
[L-18] highway_config implements a single mutable Admin while scripts/README.md asserts an immutable max-2-admin set	Resolved
[L-19] HighwayEntry::receive_message weight is benchmarked over a Linear<87,128> signer domain that diverges from the deployed 34-member committee	Resolved
[L-20] receive_message declared weight omits the dispatched payload RuntimeCall, breaking the weight upper-bound invariant	Resolved
[L-21] Payload whitelist keys on raw payload[0..2] instead of the decoded RuntimeCall identity	Resolved
[L-22] RelayerRegistry::update_bls_key overwrites a seated relayer's BLS key with no pin-window coordination, invalidating in-flight attested proofs	Acknowledged
[L-23] complete_undelegation charges the flat worst-case weight and shifts with O(n) remove	Resolved
[L-24] Proof-of-Possession reuses the message-signing ciphersuite/DST instead of a dedicated _POP_ DST	Resolved
[L-25] clear_relayer_delegations fail-fasts on one un-unbindable NFT, bricking admin recovery	Acknowledged
[L-26] A public Proof-of-Possession can be replayed to re-seat a freed BLS key onto a different relayer	Acknowledged
[L-27] Permissionless pallet_assets creation plus no asset-control binding in register_token lets a third party own a bridged asset	Resolved
[L-28] pallet_nfts bound type WeightInfo = () with permissionless CreateOrigin enables zero-weight block stuffing by any account	Resolved

[L-29] <code>remove_chain</code> becomes permanently un-callable once <code>TokenChainConfig</code> exceeds <code>RemoveChainMaxScan</code>	Acknowledged
[L-30] The <code>highway-nft-permission</code> soft-lock model has four related lock-integrity and stale-ownership gaps across <code>bind / unbind</code> and the delegation registry	Partially Resolved
[L-31] <code>clear_relayer_delegations</code> declares a fixed max-size proof above the runtime limits, making the normal admin path inadmissible	Resolved
[L-32] <code>emit_message</code> accepts an over-width EVM token target and burns funds for an unclaimable message	Acknowledged
[I-1] <code>dev_chain_spec.json</code> genesis is stale versus the audited runtime and encodes a plural <code>highwayConfig.admins</code> array	Resolved

7 Findings

7.1 Critical Risk

7.1.1 HighwayEntry::verify_bls_attestation counts all 128 relayer_bitmap bits but aggregates only the low committee_size seats

Description: verify_bls_attestation derives the claimed signer count from the entire 128-bit relayer_bitmap and enforces the signature threshold against that raw count (pallets/highway-entry/src/lib.rs:1286-1292):

```
let signer_count = relayer_bitmap.count_ones() as usize; // counts ALL 128 bits
if signer_count < min_signatures { return Err(InsufficientBlsSigners); }
if signer_count > committee_size { return Err(InvalidRelayerBitmapCommitteeOverflow); }
```

The set of public keys actually aggregated and verified is, however, built by iterating only idx in 0..committee_size (pallets/highway-entry/src/lib.rs:1335-1346):

```
for idx in 0..committee_size {
    if ((relayer_bitmap >> idx) & 1) == 0 { continue; }
    let key = T::RelayerRegistry::relayer_bls_key(committee[idx]) ...
    signer_public_keys.push(pk);
}
```

relayer_bitmap is a u128, giving 128 bit positions, but a committee has only committee_size seats. The production runtime configures CommitteeSize = 34 and MinSignatures = 24 (runtime/src/configs/mod.rs:344-345). Bit positions committee_size..128 (34..127, i.e. 94 positions) are never read by the aggregation loop and contribute no public key to the aggregate, yet they are counted by count_ones. There is no mask (no relayer_bitmap >> committee_size == 0 guard) and no rejection of out-of-range bits anywhere in the pallet: the only reads of relayer_bitmap are the weight-accounting count_ones at pallets/highway-entry/src/lib.rs:1053, the threshold count_ones at pallets/highway-entry/src/lib.rs:1286, and the aggregation-window shift (relayer_bitmap >> idx) & 1 at pallets/highway-entry/src/lib.rs:1336.

The consequence is a geometry mismatch between the counted set and the verified set. The number of public keys that must produce a valid aggregate signature is the popcount of the low committee_size bits, while the threshold is enforced against the popcount over all 128 bits. An attacker inflates the counted threshold with high junk bits that require no signature, so the number of real signers whose keys enter the aggregate can be driven far below MinSignatures while the count check still passes.

The pallet's own integrity_test asserts MinSignatures >= 87 (pallets/highway-entry/src/lib.rs:748-752) - a strong threshold on a 128-member committee, the regime in which every counted bit falls inside the aggregation window and the bug is invisible. The deployed runtime instead configures CommitteeSize = 34, MinSignatures = 24, i.e. committee_size < 128, exactly the regime where the 94 out-of-window bit positions exist and the defect is live. The entry pallet's unit-test mock uses CommitteeSize = 128, MinSignatures = 87 (pallets/highway-entry/src/mock.rs:119-120), so the test suite exercises only the full-width committee and cannot observe this defect.

Files:

- HighwayEntry::verify_bls_attestation
- HighwayEntry::receive_message

Impact: Complete collapse of the inbound signature threshold. A single malicious or compromised active committee member (out of 34) - rather than the intended 24 - can attest arbitrary inbound messages. Because inbound receive_message mints on MINT corridors and releases from escrow on RELEASE corridors, the attacker directs the minted or released funds to any account they choose, giving direct, unbounded theft of bridged funds and unbacked minting. The honest-committee threshold the bridge's security rests on (a documented 24-of-34 honest-majority assumption for inbound delivery) is silently unenforced: the code does not even enforce the signer count it claims to enforce.

Proof of Concept: Production CommitteeSize = 34, MinSignatures = 24 (runtime/src/configs/mod.rs:344-345). A single committee member suffices:

1. The attacker is any registered relayer able to submit `receive_message` - the submitter's own `relayer_id` seeds committee selection via `select_committee` (`pallets/highway-entry/src/lib.rs:1324-1330`). The attacker obtains one valid attestation signature from a single committee member occupying seat `s` ($0 \leq s < 34$) over the `BlsAttestationPreimage` for the target message (`pallets/highway-entry/src/lib.rs:1358-1367`). The BLS aggregate over a single pubkey is just that member's signature.
2. The attacker builds `relayer_bitmap = (1 << s) OR'd with 23 junk bits chosen from positions 34..127` (for example bits 34..56). Now `relayer_bitmap.count_ones() == 1 + 23 == 24`.
3. At `pallets/highway-entry/src/lib.rs:1286-1292`, `signer_count == 24`: it passes the `>= min_signatures` check ($24 \geq 24$) and the `<= committee_size` check ($24 \leq 34$).
4. The aggregation loop `idx` in `0..34` (`pallets/highway-entry/src/lib.rs:1335-1346`) sees only bit `s` set among the low 34 positions, so it reads exactly one pubkey (`committee[s]`'s registry key). The 23 junk bits at positions 34..56 are outside the loop bound and are never dereferenced.
5. `signature.fast_aggregate_verify(true, &bls_payload, DST, &[member_s_pubkey])` (`pallets/highway-entry/src/lib.rs:1373-1378`) returns `BLST_SUCCESS`, because the supplied signature is member `s`'s single signature over `bls_payload` and the aggregate contains exactly that one key.
6. `verify_bls_attestation` returns `Ok(relayer_id)` (`pallets/highway-entry/src/lib.rs:1383`); `receive_message` proceeds to `resolve_inbound_transfer` and then mints or releases the attested amount of the bridged token to `token_target_address` (`pallets/highway-entry/src/lib.rs:1092-1197`).

The intended 24-of-34 threshold is reduced, in the limit, to 1-of-34: the low bits need carry only as many real signers as form any aggregate the attacker can produce, and a single colluding or compromised committee member is enough.

Textual Step-by-Step Proof:

This proof shows how a single active committee member forges a valid inbound attestation on the deployed runtime, bypassing the intended 24-of-34 signature threshold. Every state variable, config constant, and call is cited to source. The BLS-library behavior that makes the aggregate verify against a single real signer is grounded in the resolved `blst-0.3.16` crate source.

1. Initial state.

The trigger requires the deployed FRAME runtime's committee geometry, where `CommitteeSize < 128`:

- `CommitteeSize = 34` and `MinSignatures = 24` (`runtime/src/configs/mod.rs:344-345`), wired into `pallet_highway_entry::Config` and read inside `verify_bls_attestation` as `let committee_size = T::CommitteeSize::get() as usize` and `let min_signatures = T::MinSignatures::get() as usize` (`pallets/highway-entry/src/lib.rs:1282-1283`).
- `relayer_bitmap` is typed `u128` on both the dispatchable (`pallets/highway-entry/src/lib.rs:1070`) and the helper (`pallets/highway-entry/src/lib.rs:1255`), giving 128 bit positions while the committee has only `committee_size = 34` seats.
- The active relayer set for the pinned epoch must satisfy `n_active >= committee_size` (`pallets/highway-entry/src/lib.rs:1312-1315`); the deployed floor is 34. The registry (`pallet_highway_registry`) holds each active relayer's BLS G1 pubkey, read at `pallets/highway-entry/src/lib.rs:1341` via `T::RelayerRegistry::relayer_bls_key`.
- The runtime's `integrity_test` asserts `MinSignatures >= 87` and `CommitteeSize <= 128` (`pallets/highway-entry/src/lib.rs:748-757`). The deployed `MinSignatures = 24` violates the first assertion; the config lives entirely in the `committee_size < 128` regime where the 94 out-of-window bit positions (34..127) exist. The pallet's own unit-test mock instead sets `CommitteeSize = 128`, `MinSignatures = 87` (`pallets/highway-entry/src/mock.rs:119-120`), so the full-width committee it exercises never has an out-of-window bit and cannot observe this defect.

2. Setup.

The attacker is any single active committee member (or a party who has compromised one member's operational + BLS signing keys). No admin role, no sudo, no configuration mutation is needed - `receive_message`

is ensure_signed-permissionless as to submitter identity (pallets/highway-entry/src/lib.rs:1072; ActiveRelayerCommittee is COMPETING per the Pallet and Origin Map), and the submitter's own relayer_id seeds committee selection through select_committee(&epoch_randomness, relayer_id, slot_number, &active_relayers, committee_size) (pallets/highway-entry/src/lib.rs:1324-1330). The attacker:

- Registers/controls a relayer that is active in the target epoch, so relayer_id_by_operational_key resolves the submitter (pallets/highway-entry/src/lib.rs:1295-1302) and is_active_relayer_at_epoch passes (pallets/highway-entry/src/lib.rs:1318-1320).
- Computes the deterministic committee for a chosen slot_number (0.4, pallets/highway-entry/src/lib.rs:1245-1246, 1258) seeded by their own relayer_id. select_committee returns a Vec<RelayerId> of length exactly committee_size = 34 (loop for seat in 0..committee_size, pallets/highway-entry/src/lib.rs:1425-1451), so the attacker knows which registered relayer occupies each seat 0..34.
- Identifies one committee seat s ($0 \leq s < 34$) whose occupant will produce a signature - in the simplest case, seat s is the attacker's own relayer, so no external signature is needed at all; the coalition is 1-of-34. The single signer signs the domain-separated BlsAttestationPreimage (domain = b"HWY_BLS_V1", network_id, destination_chain_id = LocalChainId = 2, message_id, ttl, slot_number, relayer_id) for the target inbound message (pallets/highway-entry/src/lib.rs:1358-1367). A BLS aggregate over a single pubkey is just that member's own signature.

3. Trigger.

The attacker calls HighwayEntry::receive_message(...) (pallets/highway-entry/src/lib.rs:1054-1071) with the fields for the target inbound transfer (a MINT or RELEASE corridor message crediting token_target_address), passing bls_signature = the single-signer signature from step 2 and a crafted relayer_bitmap:

- relayer_bitmap = (1u128 << s) | junk, where junk is 23 bits chosen from the out-of-window positions 34..127 - e.g. bits 34,35,...,56. Then relayer_bitmap.count_ones() == 1 + 23 == 24.

Inside verify_bls_attestation, the two independent uses of the bitmap diverge geometrically:

- Threshold check (counts ALL 128 bits): let signer_count = relayer_bitmap.count_ones() as usize yields signer_count == 24 (pallets/highway-entry/src/lib.rs:1286). The guard if signer_count < min_signatures is $24 < 24 = \text{false}$ (passes, pallets/highway-entry/src/lib.rs:1287-1289); the guard if signer_count > committee_size is $24 > 34 = \text{false}$ (passes, pallets/highway-entry/src/lib.rs:1290-1292). No mask, no (relayer_bitmap >> committee_size) == 0 guard, and no out-of-range rejection exists anywhere in the pallet - the only reads of relayer_bitmap are the weight-accounting count_ones at pallets/highway-entry/src/lib.rs:1053, the threshold count_ones at pallets/highway-entry/src/lib.rs:1286, and the aggregation shift at pallets/highway-entry/src/lib.rs:1336.
- Aggregation window (reads ONLY the low committee_size bits): for idx in 0..committee_size = for idx in 0..34, with if ((relayer_bitmap >> idx) & 1) == 0 { continue; } (pallets/highway-entry/src/lib.rs:1335-1338). Among positions 0..33 only bit s is set, so the loop pushes exactly one pubkey: committee[s]'s registry key (pallets/highway-entry/src/lib.rs:1340-1345). The 23 junk bits at 34..56 lie at or above the loop bound committee_size = 34 and are never shifted-in, never index committee[], and never dereference a registry key.

This is the bit-count vs low-committee_size-aggregate divergence: the counted set has 24 members, the aggregated set has 1.

4. Resulting state.

- signer_public_keys contains exactly one key (committee[s]'s), pallets/highway-entry/src/lib.rs:1333-1346. The aggregate handed to BLS is a single-key aggregate.
- signature.fast_aggregate_verify(true, &bls_payload, DST, &signer_pk_refs) returns BLST_ERROR::BLST_SUCCESS (pallets/highway-entry/src/lib.rs:1373-1381). Grounding the external behavior in the resolved crate: blst's fast_aggregate_verify aggregates only the supplied pks slice into one aggregate public key via AggregatePublicKey::aggregate(pks, false) and then verifies msg

against that single aggregate (blst-0.3.16/src/lib.rs:1275-1294). The library has no notion of the bitmap or committee size - it sees only the one pubkey the loop pushed, and the supplied signature is exactly that member's signature over bls_payload, so the pairing check succeeds. The 23 junk bits are invisible to blst.

- `verify_bls_attestation` returns `Ok(relayer_id)` (pallets/highway-entry/src/lib.rs:1383) despite only 1 real signer against a nominal 24-signer threshold.
- Control returns to `receive_message`, which records the message as executed (`ExecutedInboundMessages::<T>::insert`, pallets/highway-entry/src/lib.rs:1146) and dispatches `resolve_inbound_transfer -> execute_inbound_native_operation / execute_inbound_asset_operation` (pallets/highway-entry/src/lib.rs:1148-1197). On a MINT corridor this mints the attested amount of the bridged token to `token_target_address`; on a RELEASE corridor it releases that amount from escrow to `token_target_address`. Net changed outcome: an inbound message backed by a single forged-threshold attestation clears every check the threshold logic claims to enforce and pays out funds.

5. Impact quantification.

Complete collapse of the inbound signature threshold. The intended honest-majority gate for inbound delivery is 24-of-34 (`CommitteeSize = 34`, `MinSignatures = 24`, `runtime/src/configs/mod.rs:344-345`); the deployed code reduces it, in the limit, to 1-of-34. A single malicious or compromised active committee member can attest arbitrary inbound messages, choosing `token_target_address` and amount freely.

The authorization breach is exact: `verify_bls_attestation` does not enforce the signer count it claims to enforce - it counts 24 but verifies against 1. In FRAME accounting terms the payout path is unconstrained by any independent rate limit, withdrawal delay, or circuit breaker (only the per-corridor [`min_amount`, `max_amount`] bounds apply), so a forged attestation on a MINT corridor issues unbacked bridged tokens directly to the attacker, and on a RELEASE corridor drains escrowed collateral up to the escrow balance, repeatable per distinct `message_id`.

Attack cost vs payoff: cost is control of one committee seat out of 34 (one compromised or self-owned relayer's operational + BLS keys) plus the ordinary extrinsic fee/nonce for `receive_message` - the submitter pays only gas-equivalent transaction fees (`WeightToFee`, `runtime/src/lib.rs:123-131`). Payoff is arbitrary, unbounded issuance/release of bridged assets to an attacker-chosen account - direct theft of bridged funds and unbacked minting. The 23 junk high bits carry no signature, cost nothing to set, and are the entire exploit primitive.

Recommended Mitigation: Bind the counted set to the aggregated set so `signer_count` equals the number of pubkeys the loop aggregates. In `verify_bls_attestation`, immediately before `let signer_count = ...` at `pallets/highway-entry/src/lib.rs:1286`, either:

(a) reject any out-of-range bit outright (preferred):

```
ensure!(
    committee_size >= 128 || (relayer_bitmap >> committee_size) == 0,
    Error::::InvalidRelayerBitmapCommitteeOverflow
);
```

or (b) count only the low `committee_size` bits:

```
let mask = if committee_size >= 128 { u128::MAX } else { (1u128 << committee_size) - 1 };
let signer_count = (relayer_bitmap & mask).count_ones() as usize;
```

With either fix the threshold is enforced against the real signers actually aggregated. The weight annotation at `pallets/highway-entry/src/lib.rs:1053` also reads `count_ones` over the full bitmap; masking there as well keeps the weight index inside the benchmarked range. Separately, reconcile the deployed `MinSignatures = 24` against the `integrity_test` assertion `MinSignatures >= 87` (`pallets/highway-entry/src/lib.rs:748-752`), which the current runtime config violates.

Highway: Fixed in [c66a229](#), [2701c27](#), [3cd5d6e](#).

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 `NftDelegationRegistry::delegate` Path A never binds the operator NFT caller to the relayer's manager

Description: `NftDelegationRegistry::delegate` Path A (the `tier.is_relayer_tier()` branch, `pallets/highway-nft-delegation-registry/src/lib.rs:330-434`) establishes the on-chain proof that a relayer node is backed by an NFT: it sets `RelayerOperatorTier[relayer_id]`, which becomes the relayer's capacity ceiling and tier. The only guards on Path A are that `relayer_id` is registered (`pallets/highway-nft-delegation-registry/src/lib.rs:337-340`), the caller's NFT is a relayer tier, `RelayerOperatorTier[relayer_id]` is currently `None` (`RelayerAlreadyBacked`, `pallets/highway-nft-delegation-registry/src/lib.rs:353-356`), and the caller owns the NFT. There is no check that the caller is the relayer's operator or manager. The registry's `Relayer::manager` field is documented as informational only, carrying no on-chain privileges, and `delegate` never consults it or the relayer's operational keys. The binding between "the account that operates `relayer_id`" and "the account that supplies `relayer_id`'s backing NFT" is entirely absent.

Consequently the first relayer-tier-NFT holder to call `delegate(nft, relayer_id)` for a given registered `relayer_id` wins that relayer's backing slot until it is undelegated, even with no relationship to that relayer. Relayer-tier NFTs are held by every relayer operator and are transferable, so the attacker pool is the entire operator set.

Files:

- `NftDelegationRegistry::delegate`

Impact: A relayer's operator backing and capacity ceiling - security-relevant state meant to represent the operator's own NFT stake - can be set by an unrelated party who merely holds any relayer-tier NFT. Concrete harms: (a) denial of service of a legitimate operator establishing on-chain backing for their own relayer (they hit `RelayerAlreadyBacked` indefinitely); (b) capacity throttling, since the attacker's low-tier NFT caps how much delegator weight the relayer can accept, starving it of delegations; (c) the attacker can, at any time, `undelegate + complete_undelegation` the backing NFT, which for a relayer-tier NFT runs `RelayerOperatorTier::remove(relayer_id)` and `deauthorize_operator(relayer_id)`, clearing the relayer's authorization flag - an authority state the attacker never had rights to touch. This is a permissionless hijack of authority-conferring state with no third-party consent, though the attacker pool is the (admin-curated) set of relayer-tier-NFT holders rather than the open public, and admin recovery via `clear_relayer_delegations` exists.

Recommended Mitigation: On Path A `delegate`, require the caller to be a registered controller of `relayer_id`. Options: add an on-chain, privilege-bearing operator/manager binding in the registry and check it; or gate Path A behind `is_authorized_operator(relayer_id)` having been set by admin for this operator; or record the intended operator `AccountId` at `authorize_operator` time and require `who == recorded_operator` in `delegate`. Whichever is chosen, the account that supplies a relayer's backing NFT must be provably the operator of that relayer. Present tier and capacity checks are correct and should be preserved.

Highway: Fixed in [0638764](#), [5fd9a6f](#), [5de2041](#).

Cyfrin: Verified.

7.2.2 `DefaultUndelegationCooldown=0` collapses the slash window between `undelegate` and `complete_undelegation`

Description: The delegation registry locks an NFT while it is delegated and gates its release behind an undelegation cooldown. The pallet doc comment states the cooldown's purpose: the NFT "remains locked (transfer disabled) until `complete_undelegation` is called after the cooldown, preventing a sale before any potential slashing action" (a "15-day undelegation cooldown for stability"). Slashing (`slash_delegation`) can only act on a live Delegations record; `complete_undelegation` removes that record, after which `slash_delegation` reverts `Nft-NotDelegated`. The cooldown is therefore the only thing that keeps a misbehaving operator's or delegator's stake exposed to a slash long enough for governance to act.

The cooldown value comes from `UndelegationCooldown`, whose on-empty default is `T::DefaultUndelegationCooldown`. The runtime wires `DefaultUndelegationCooldown = 0` (`runtime/src/configs/mod.rs:535`), and no genesis preset or deployment script sets `UndelegationCooldown`, so the operational value is 0. With a zero

cooldown, undelegate computes `completion_block = current_block.saturating_add(0) = current_block` (pallets/highway-nft-delegation-registry/src/lib.rs:457-458), and `complete_undelegation`'s guard `current_block >= completion_block` (pallets/highway-nft-delegation-registry/src/lib.rs:476-521, guard at 483-486) is satisfied in the same block. Because these are two separate extrinsics, they can be bundled in a single `pallet_utility::batch`, making the entire escape one atomic call. Raising the cooldown later does not retroactively lengthen an already-recorded `completion_block`, so the cooldown is non-retroactive.

Files:

- `Pallet::undelegate`
- `Pallet::complete_undelegation`

Impact: The slashing deterrent that backs relay honesty is neutralized as deployed. An operator or delegator who observes (or front-runs) a pending `slash_delegation` can undelegate then `complete_undelegation` in one block, unbind the relay-tier NFT (re-enabling transfer), drop its weight from the relay, and transfer or sell the NFT before the slash lands. The subsequent `slash_delegation` reverts `NftNotDelegated`, so the value can no longer be slashed. No third-party funds are directly stolen and the value at risk is bounded to the operator's own NFT weight (an off-chain reward-share weight, not a token balance), but the economic accountability the pallet advertises is void from launch; `set_undelegation_cooldown` can raise it later, bounding severity.

Recommended Mitigation: Set `DefaultUndelegationCooldown` (runtime/src/configs/mod.rs:535) to the intended non-zero block count (about 216000 blocks for 15 days at 6-second blocks), or seed `UndelegationCooldown` at genesis, so the shipped chain enforces the documented lock. Independently of the value, do not let a delegator-controlled call destroy slashable exposure while the protocol still needs it: for example, reject `complete_undelegation` while a slash or challenge is in progress, or retain the zeroed `Delegations` record for a slash-only grace period after completion so a slash can still land. If a zero cooldown is genuinely intended for some deployments, remove the "15-day cooldown" / "preventing a sale before any potential slashing action" documentation so the guarantee is not falsely advertised.

Highway: Acknowledged. The undelegation cooldown is configurable via `set_undelegation_cooldown`, and the deployed default is 0, which provides no slash window; the module and `undelegate` documentation are corrected to state this rather than advertise a 15-day guarantee. Under the current MVP model there is no redelegate path and operator misbehaviour is handled off-chain by revoking the NFT, so on-chain slashing is not an active economic path. If slashing is introduced later, `complete_undelegation` will be gated against a pending slash.

Cyfrin: Rationale accepted with a condition: the zero cooldown is acceptable only while on-chain slashing remains outside the MVP economic model; before relying on slashing, enforce a nonzero slash window or block completion while a slash is pending.

7.2.3 HighwayEntry::execute_inbound_asset_operation MINT to a provider-less fresh recipient can fail the pallet_assets minimum-balance check

Description: The asset inbound MINT branch calls `T::Currency::mint_into(asset_id, recipient, amount).map_err(|_| MintFailed)?` (pallets/highway-entry/src/lib.rs:2095), where `Currency` is `pallet_assets`. For a recipient with no existing account for that asset, the underlying `can_increase` check returns `BelowMinimum` when `amount < min_balance`, and `CannotCreate` when the asset is not sufficient and the recipient has no provider reference (no native existential-deposit balance and no other sufficient asset). Bridged assets in this deployment are created with `is_sufficient: false`, so every inbound asset MINT to a brand-new account with no native existential-deposit balance reverts with `CannotCreate` regardless of amount, and any MINT below the asset's `min_balance` reverts with `BelowMinimum`.

There is no on-chain coupling between the corridor `min_amount` and the asset's `min_balance`. The corridor `min_amount` is validated only as `> 0` and against the decimal-flooring rule in `configure_token_bridge` (pallets/highway-config/src/lib.rs:588-613); it is never checked against the asset's `min_balance`. An admin can therefore configure a corridor whose entire band below `min_balance` is undeliverable. The replay mark is inserted before the mint and rolls back on the revert, so the message stays retryable - but it can never succeed until the recipient independently acquires a provider reference (someone funds its native existential deposit) or the corridor `min_amount` is raised.

Files:

- `HighwayEntry::execute_inbound_asset_operation`
- `HighwayConfig::configure_token_bridge`

Impact: Inbound MINT deliveries to the most common recipient shape for a bridge - a brand-new account receiving its first bridged tokens - revert and strand the source-debited funds until an out-of-band native-existential-deposit funding of the recipient or a governance corridor reconfiguration. The recipient is chosen by the source-chain sender, so this affects ordinary first-time users, not just an attacker's own account. Recovery is source-side admin action or third-party existential-deposit funding, not permissionless. This is a bridge-liveness / delivery-failure defect on a recurring legitimate path, distinct from a reverting-payload strand (the token leg itself reverts, with no payload involved) and from config-read-at-claim drift (the config here is stable and correct-looking, yet delivery fails on the asset `min_balance` / sufficiency precondition).

Proof of Concept: Bridged asset A is created non-sufficient with `min_balance = 1_000_000`. A corridor for (A, source S) is configured `inbound = Mint`, `min_amount = 1000` (passes the `min_amount > 0` and decimal-floor validation).

Case 1 (BelowMinimum): a source user bridges a local amount of 5000 (`>= min_amount 1000`) to recipient R. `resolve_inbound_transfer` passes because `5000 >= 1000`. `execute_inbound_asset_operation` (`pallets/highway-entry/src/lib.rs:2084-2105`) calls `mint_into(A, R, 5000)`; R has no account for A, so `can_increase` sees `5000 < min_balance 1_000_000`, returns `BelowMinimum`, mapped to `MintFailed`, and `receive_message` reverts. S has already debited the sender; the message can only ever succeed if the corridor `min_amount` is raised.

Case 2 (CannotCreate): a source user bridges a local amount of `2_000_000 (>= min_balance)` to a fresh account R with zero native balance and no other assets. `mint_into(A, R, 2_000_000)` reaches `can_increase`, which finds the recipient has no provider reference and A is non-sufficient, returns `CannotCreate`, mapped to `MintFailed`, and `receive_message` reverts. The transfer cannot complete until R is funded with the native existential deposit out of band; a first-time bridge-in recipient cannot self-remediate.

Recommended Mitigation: At corridor-config time for MINT-inbound corridors, reject `min_amount < min_balance(asset)` by reading the asset's minimum balance. For the sufficiency/provider problem, either require bridged assets used on MINT corridors to be sufficient, or have the bridge pre-seed the recipient's provider reference (a minimal native deposit) before minting, or document that MINT recipients must be pre-funded. At minimum, provide an explicit admin recovery / re-mint primitive so a reverting MINT does not strand indefinitely.

Highway: Fixed in [62f62ed](#).

Cyfrin: Verified.

7.2.4 Inbound payload dispatches with the delivering relayer's Signed origin; a whitelisted Balances transfer drains the relayer

Description: The inbound payload path dispatches the decoded call with the delivering relayer's Signed origin. `receive_message` authenticates the submitter as a registered relayer (ensure_signed at [pallets/highway-entry/src/lib.rs#L1072](#), mapped to a `relayer_id` via `relayer_id_by_operational_key`, erroring `RelayerNotRegistered`, at [pallets/highway-entry/src/lib.rs#L1296-L1302](#)) and then calls `Self::execute_payload(payload_bytes, &sender)` ([pallets/highway-entry/src/lib.rs#L1210](#)). `execute_payload` ([pallets/highway-entry/src/lib.rs#L1798-L1811](#)) decodes the payload into a `RuntimeCall` and dispatches it with `RawOrigin::Signed(caller)` ([pallets/highway-entry/src/lib.rs#L1806-L1807](#)), where `caller` is that relayer:

```
fn execute_payload(payload: &[u8], caller: &T::AccountId) -> Result<(), Error<T>> {
    let call = <T as Config>::RuntimeCall::decode(&mut &payload[..])
        .map_err(|_| Error::<T>::InvalidPayloadFormat)?;
    let origin = frame_system::RawOrigin::Signed(caller.clone()).into();
    call.dispatch(origin).map_err(|_| Error::<T>::PayloadExecutionFailed)?;
    Ok(())
}
```

So any payload that decodes to an origin-sensitive call executes with the relayer's account authority.

The codebase documents two different payload origins and shipped the unsafe one. The `execute_payload` doc-comment describes the shipped path, that it "dispatches it with Signed origin using the relayer's account. The relayer who delivers the message is the caller." Yet `pallet_account_id()` ([pallets/highway-entry/src/lib.rs#L1775](#)) carries a doc comment earmarking it "to execute whitelisted cross-chain payload calls with limited privileges (not Root)", and it is dead code with no call site (its only occurrence in the file is the definition). A neutral, keyless payload origin was therefore specced but never wired up, and the path that shipped runs under the relayer instead. The live origin is the relayer.

The only payload control is the `(pallet_index, call_index)` whitelist (`extract_payload_indices` at [pallets/highway-entry/src/lib.rs#L1813](#), `ensure_payload_whitelisted` at [pallets/highway-entry/src/lib.rs#L1821](#)), which inspects the raw two-byte prefix, not the dispatch origin. So an origin-funded call such as `Balances::transfer_all` presents as an ordinary, allowable `Balances` entry.

Impact: A source-chain attacker emits a message whose payload is an origin-sensitive call, for example `Balances::transfer_all { dest: attacker, keep_alive: false }`. The committee attests it (committees attest valid source emissions; they do not judge payload safety), and any registered relayer that delivers it through `receive_message` has the call dispatched as `Signed(itself)`, transferring the relayer's entire free balance to the attacker. `transfer_all` takes no amount (the attacker need not know the balance) and `keep_alive: false` reaps the account.

The victim is the honest delivering relayer: a non-relayer source-chain actor borrows relayer authority (privilege escalation), and any relayer-gated whitelisted call is reachable the same way. Each drain costs one emission and is repeatable across the relayer set, so once such a call is whitelisted the relayers can be drained systematically. The whitelist is the only barrier, and a generic `Balances` transfer is exactly what a reviewer would deem safe to allow; the intended neutral-origin defense (`pallet_account_id`, dead code) leaves no defense in depth.

As shipped, the whitelist is `System::remark / remark_with_event` only ([scripts/constants.js#L56-L58](#)), so no funds- or role-gated call is currently whitelisted and the drain is not realizable on the deployed configuration. It becomes a full relayer drain the moment an admin whitelists any funds- or role-gated call, which for a token bridge (whose purpose is to deliver value-bearing calls) is a plausible production step rather than an exotic misconfiguration. So this is a latent confused-deputy defect: the payload borrows the relayer's own authority, one whitelist entry away from realized fund loss, and the neutral-origin fix removes that exposure regardless of what is whitelisted.

Impact worsens if a relayer reuses one account as both its operational key and its value-bearing / NFT-holding account, which nothing forces to be distinct (Path A "self-bind" delegates the operator's own NFT). The drain then hits real funds, and any whitelisted `pallet_nfts` transfer/delegation call reaches the backing NFT itself (subject to the transfer-lock, which has known gaps). Safety rests on an unstated, unenforced assumption that operators isolate an empty operational key from their funds and NFT; the neutral-origin fix removes that dependency.

Proof of Concept: Runnable test `payload_dispatched_as_relayer_drains_the_delivering_relayer` in `pallets/highway-entry/src/tests.rs`. It funds a relayer (mock account 1, genesis balance 1_000_000), builds the `transfer_all` payload, then reproduces `execute_payload`'s two operations (decode + dispatch as `Signed(relayer)`) against the real mock runtime with `pallet_balances`, and asserts the relayer ends at 0 while the attacker holds the relayer's full 1_000_000.

```
#[test]
fn payload_dispatched_as_relayer_drains_the_delivering_relayer() {
    use codec::{Decode, Encode};
    use polkadot_sdk::frame_system::RawOrigin;
    use polkadot_sdk::sp_runtime::traits::Dispatchable;

    new_test_ext().execute_with(|| {
        let relayer: u64 = 1; // a registered relayer; genesis-funded with 1_000_000
        let attacker: u64 = 42; // attacker-controlled sink; starts empty

        let relayer_start = Balances::free_balance(&relayer);

        // Innocent-looking `Balances` call; transfer_all takes the origin's entire free balance.
        let malicious_call = RuntimeCall::Balances(mock::pallet_balances::Call::transfer_all {
            dest: attacker,
            keep_alive: false,
        });
    });
}
```

```

    let payload = malicious_call.encode();

    // exactly what execute_payload(payload, &relayer) does (lib.rs:1798-1811):
    let decoded = RuntimeCall::decode(&mut &payload[..]).expect("decodes");
    assert_ok!(decoded.dispatch(RawOrigin::Signed(relayer).into()));

    assert_eq!(Balances::free_balance(&relayer), 0); // relayer drained
    assert_eq!(Balances::free_balance(&attacker), relayer_start); // to the attacker
  });
}

```

Run it from the workspace root:

```
cargo test -p pallet-highway-entry payload_dispatched_as_relayer_drains_the_delivering_relayer
```

Result: test tests::payload_dispatched_as_relayer_drains_the_delivering_relayer ... ok (relayer balance 1_000_000 to 0, attacker 0 to 1_000_000).

Recommended Mitigation: Dispatch payloads from the pallet's own deterministic, keyless account, `pallet_account_id()`, instead of the relayer. The pallet already documents this account as the intended payload-dispatch origin ("execute whitelisted cross-chain payload calls with limited privileges (not Root)") but left it as dead code; the fix is to wire it up. Being `PalletId`-derived it has no private key, so only the runtime can ever act as it, which removes the borrowed-authority drain. This is sound only as long as `pallet_account_id()` is kept fund-less and role-less: it must never be configured as an Escrow/Release account or a fee collector, and never be granted a role, so that an origin-funded call such as `Balances::transfer_all` moves nothing and any role-gated call fails `BadOrigin`. Then whitelist only origin-agnostic calls, never one that moves the origin's funds or is gated on the caller's identity or role. And never whitelist a wrapper call (`Utility::batch` / `batch_all` / `force_batch` / `as_derivative`, plus any future `proxy` / `multisig`): the whitelist checks only the outer (`pallet_index`, `call_index`), so a whitelisted wrapper smuggles arbitrary unchecked inner calls under the same origin.

```

use polkadot_sdk::sp_runtime::traits::AccountIdConversion;

// `pallet_account_id()` already exists and is documented as the intended
// payload-dispatch origin, but is currently dead code. Wire it up. It is a
// deterministic, keyless PalletId-derived account; it MUST be kept fund-less and
// role-less (never an Escrow/Release/fee-collector account, never granted a role).
fn pallet_account_id() -> T::AccountId {
    T::PalletId::get().into_account_truncating()
}

// `caller` (the delivering relayer) is still authenticated in `receive_message`
// for relayer-gating / replay, but it is NO LONGER the dispatch origin.
fn execute_payload(payload: &[u8]) -> Result<(), Error<T>> {
    let call = <T as Config>::RuntimeCall::decode(&mut &payload[..])
        .map_err(|_| Error::<T>::InvalidPayloadFormat)?;

    // Dispatch from the pallet's keyless account, NOT the relayer. With no funds
    // and no roles, `Balances::transfer_all` moves nothing and any role-gated
    // call fails `BadOrigin`, so a borrowed-authority payload is inert.
    let origin = frame_system::RawOrigin::Signed(Self::pallet_account_id()).into();
    call.dispatch(origin).map_err(|_| Error::<T>::PayloadExecutionFailed)?;
    Ok(())
}

```

The relayer is only removed as the dispatch origin; `receive_message`'s relayer authentication and replay protection are unchanged. If you would rather not reuse the pallet's sovereign account for dispatch (in case it is ever also an escrow or fee account), derive a dedicated, guaranteed-distinct sub-account instead, `T::PalletId::get().into_sub_account_truncating(b"hwpy/pyld")`, the strictly more defensive variant.

Highway: Fixed in [83a41a0](#).

Cyfrin: Verified.

7.2.5 Caller-chosen relay ID and slot amplify a sub-third coalition's committee-takeover probability by over 9,500x

Description: Inbound messages are authorized by a sampled committee of `CommitteeSize` relayers, of which `MinSignatures` must sign. The runtime sets `CommitteeSize = 34` and `MinSignatures = 24` (runtime/src/configs/mod.rs:343-345), a documented 2/3-plus supermajority ($\text{ceil}(2 * 34 / 3) + 1 = 24$). For any already-selected committee containing at most 11 malicious members, the 24-signature threshold gives a deterministic safety property. However, controlling fewer than one third of the larger active set does not deterministically bound the malicious fraction of a randomly sampled committee: even one honest draw has a small probability of selecting at least 24 malicious members.

The defect is that committee selection is not one honest draw. After the epoch randomness is public, the submitter can adaptively choose two seed inputs and select the most favorable result. The seed is (pallets/highway-entry/src/lib.rs:1386-1398):

```
seed = keccak256(epoch_randomness || relayer_id_LE || slot_number_LE)
```

- `epoch_randomness` is fixed and public for the epoch. This finding assumes it is honestly generated and unbiased; the attack does not require influencing it. The value is supplied when an admin or authorized updater installs the active set (pallets/highway-registry/src/lib.rs:1380-1413, 1908-1921).
- `relayer_id` is the submitter's relayer identity. `verify_bls_attestation` maps the signed origin to a `relayer_id` (pallets/highway-entry/src/lib.rs:1294-1302) and only requires that the ID is active in the supplied epoch (pallets/highway-entry/src/lib.rs:1317-1320). A coalition controlling `m` active operational keys can therefore choose among its `m` IDs.
- `slot_number` is caller-supplied and only range-checked to 0..5 (pallets/highway-entry/src/lib.rs:1257-1259).

Both attacker-chosen values flow into `select_committee` (pallets/highway-entry/src/lib.rs:1322-1330), which deterministically samples a 34-seat committee by rejection sampling over the active set (pallets/highway-entry/src/lib.rs:1407-1454). A coalition controlling `m` active relayer identities therefore has $5 * m$ distinct seed choices after seeing the epoch randomness. It can compute every corresponding committee draw locally and submit using the (`relayer_id`, `slot_number`) pair that gives it the most seats. Nothing assigns a message an objective submitter or slot, so the verifier accepts an adaptive best-of- $5m$ search instead of one protocol-assigned draw.

Binding `relayer_id` and `slot_number` into the signed `BlsAttestationPreimage` (pallets/highway-entry/src/lib.rs:1358-1367) does not prevent the attack. After choosing the favorable pair, the coalition creates the aggregate for that same pair using the genuine keys of the selected malicious committee members.

The favorable committee is reusable for the remainder of the epoch because `message_id` is included in the BLS payload but absent from the committee seed and `select_committee` (seed at pallets/highway-entry/src/lib.rs:1386-1398; selection at pallets/highway-entry/src/lib.rs:1407-1454). Replay protection is per message ID (`ExecutedInboundMessages` at pallets/highway-entry/src/lib.rs:1076-1080, 1146), so the coalition can vary source nonce/block fields, recompute a fresh canonical ID as `receive_message` does (pallets/highway-entry/src/lib.rs:1102-1132), sign it with the same malicious committee, and repeat.

Files:

- `HighwayEntry::verify_bls_attestation`
- `HighwayEntry::select_committee` / `HighwayEntry::committee_seed`
- `HighwayEntry::receive_message`

Impact: When a sub-third active-set coalition finds a draw in which it occupies at least 24 of the 34 committee seats, it can authenticate nonexistent source transfers. On a MINT corridor this mints unbacked assets (pallets/highway-entry/src/lib.rs:2092-2105); on a RELEASE corridor it drains escrowed collateral to attacker accounts (pallets/highway-entry/src/lib.rs:2106-2115; native paths at pallets/highway-entry/src/lib.rs:2038-2069). The attack uses genuine registered keys, genuine signatures, and only low, in-range bitmap positions, so the BLS, signer-count, and bitmap checks all pass.

The runtime can represent relay IDs 1..=6000 through `MaxBitmapSize = 750` and `MAX_RELAYER_ID_CAP = 6000` (runtime/src/configs/mod.rs:440; pallets/highway-registry/src/lib.rs:193-194). This is a maximum-capacity scenario, not evidence that 6,000 relayers are currently active. At that capacity, a coalition controlling 1,999 of 6,000 active relayers has the following probabilities:

one unbiased committee draw:	$p = 9.3293204722e-6$	(~0.000933%)
9,995 caller-selected draws:	$P = 0.0890315304$	(~8.903%)
amplification:	$P / p \approx 9,543x$	

The likelihood depends materially on the actual active-set population. The following sensitivity table uses the largest coalition strictly below one third, $m = \text{floor}((N - 1) / 3)$, and models its 5m distinct Keccak seeds as independent pseudorandom committee draws:

Active relayers N	Coalition m	Success per epoch	Mean wait at one-hour epochs
128	42	0.00241%	4.7 years
256	85	0.0774%	53.8 days
500	166	0.357%	11.7 days
6,000	1,999	8.903%	11.2 hours

These estimates assume independent epoch randomness and a stable coalition. The deterministic vector in the PoC proves that a favorable maximum-capacity draw exists independently of the probability approximation.

Severity rationale (why Medium, not High). The impact is high, but exploitation requires control of a coalition close to one third of the active set and its likelihood is population-dependent. No privileged call is needed at exploitation time if the attacker controls the operational and BLS keys of already-active relayers. If the attacker instead tries to build the coalition through Sybil identities, registration requires an admin or authorized registrar (pallets/highway-registry/src/lib.rs:763-785), and inclusion in the active set separately requires an admin or authorized updater (pallets/highway-registry/src/lib.rs:1380-1413, 1908-1921); a registrar alone cannot activate the identities. If production operates a large active set and explicitly promises safety against every sub-third coalition, the severity should be revisited upward.

Proof of Concept: The following maximum-capacity scenario uses the runtime's `CommitteeSize = 34` and `MinSignatures = 24` (runtime/src/configs/mod.rs:343-345) and assumes an honestly generated public epoch randomness. An attacker controls already-active relay IDs 1..=1999, which is strictly less than one third of the 6,000-member active set:

1. After the epoch randomness becomes public, the attacker locally evaluates `select_committee` for every controlled `relay_id` and each `slot_number` in 0..5, giving $5 * 1999 = 9995$ distinct seed choices.
2. For `epoch_randomness = 0x47c8aa2c166ef35f89a1f672fd425a2594b9b08b1cfa4f1693e217cb5b1a9832`, (`relay_id = 1320`, `slot_number = 1`) yields a committee with exactly 24 attacker-controlled seats.
3. The attacker sets those 24 low, in-range bitmap positions and produces 24 genuine signatures over the target message's `BlsAttestationPreimage`.
4. Submission from relay 1320's operational key satisfies the signer-count check (pallets/highway-entry/src/lib.rs:1282-1292), key collection (pallets/highway-entry/src/lib.rs:1332-1346), and aggregate verification (pallets/highway-entry/src/lib.rs:1358-1381).

For one uniformly sampled 34-seat committee, the probability that at least 24 seats belong to the coalition is the hypergeometric tail:

$p = \sum_{i=24..34} C(1999, i) * C(4001, 34 - i) / C(6000, 34)$ $\approx 9.3293204722e-6$
--

Modeling the 9,995 distinct seeds as independent pseudorandom draws:

```
P(at least one favorable draw in an epoch)
= 1 - (1 - p)^9995
~= 0.0890315304 (~8.903%)
```

The following passing selector-level test invokes the pallet's exact committee-selection implementation. Paste it into the existing `committee_selection` module in `pallets/highway-entry/src/tests.rs`:

```
#[test]
fn subthird_caller_slot_grinding_reaches_runtime_quorum() {
    new_test_ext().execute_with(|| {
        let active_relayers: Vec<u32> = (1..=6_000).collect();
        assert!(1_999usize * 3 < active_relayers.len());

        let epoch_randomness = [
            0x47, 0xc8, 0xaa, 0x2c, 0x16, 0x6e, 0xf3, 0x5f,
            0x89, 0xa1, 0xf6, 0x72, 0xfd, 0x42, 0x5a, 0x25,
            0x94, 0xb9, 0xb0, 0x8b, 0x1c, 0xfa, 0x4f, 0x16,
            0x93, 0xe2, 0x17, 0xcb, 0x5b, 0x1a, 0x98, 0x32,
        ];

        let committee = HighwayEntry::select_committee_for_tests(
            &epoch_randomness,
            1_320,
            1,
            &active_relayers,
            34,
        )
        .unwrap();

        assert_eq!(
            committee,
            vec![
                1719, 369, 3436, 1300, 1292, 107, 293, 3200, 746, 497,
                3560, 1856, 1603, 5523, 1357, 965, 2911, 1831, 3535, 1880,
                3687, 4626, 189, 2002, 1675, 1542, 19, 863, 958, 1007,
                5851, 393, 1184, 1421,
            ]
        );

        let attacker_positions: Vec<usize> = committee
            .iter()
            .enumerate()
            .filter_map(|(position, relay_id)| (*relay_id <= 1_999).then_some(position))
            .collect();
        assert_eq!(attacker_positions.len(), 24);
        assert!(attacker_positions.iter().all(|position| *position < 34));

        let relay_bitmap = attacker_positions
            .iter()
            .fold(0u128, |bitmap, position| bitmap | (1u128 << *position));
        assert_eq!(relay_bitmap.count_ones(), 24);
        assert_eq!(relay_bitmap >> 34, 0);
    });
}
```

Run:

```
cargo test -p pallet-highway-entry subthird_caller_slot_grinding_reaches_runtime_quorum -- --nocapture
```

Expected result:

```
running 1 test
```

```
test tests::committee_selection::subthird_caller_slot_grinding_reaches_runtime_quorum ... ok
test result: ok. 1 passed; 0 failed
```

This is deliberately a selector-level PoC: it proves that the exact on-chain selection algorithm produces a valid 24-seat malicious quorum using only low bitmap positions. It does not instantiate 6,000 registry entries, construct the 24 BLS signatures, or execute a mint. Downstream feasibility follows from the coalition owning those 24 genuine keys and from the verifier collecting and checking exactly the bitmap-selected committee keys (pallets/highway-entry/src/lib.rs:1282-1381).

Textual Step-by-Step Proof:

1. Initial state.

- The runtime reads `CommitteeSize = 34` and `MinSignatures = 24` in `verify_bls_attestation` (runtime/src/configs/mod.rs:343-345; pallets/highway-entry/src/lib.rs:1282-1283).
- In the maximum-capacity scenario, active relayer IDs are `1..=6000`; the attacker controls the operational and BLS keys for IDs `1..=1999`. This is an assumed reachable runtime state, not a claim about the currently deployed active population.
- The epoch randomness is public and fixed for `DefaultEpochDurationBlocks = 600`, roughly one hour (runtime/src/configs/mod.rs:441). The attack assumes honest randomness and does not require updater authority.

2. Grinding.

- The attacker evaluates the seed `keccak256(epoch_randomness || relayer_id_LE || slot_number_LE)` for all 9,995 controlled `(relayer_id, slot_number)` pairs (pallets/highway-entry/src/lib.rs:1386-1398).
- For the PoC randomness, `(1320, 1)` selects the 34-seat vector above, with exactly 24 IDs in the attacker's set.
- The attacker maps those seats to 24 low bitmap bits and signs the target `BlsAttestationPreimage` with the corresponding genuine keys (pallets/highway-entry/src/lib.rs:1358-1367).

3. Trigger.

- The attacker chooses an active source chain, configured token corridor, attacker-controlled recipient, and in-bounds amount, then computes the canonical `V1 message_id` from arbitrary source block/nonce fields as `receive_message` does (pallets/highway-entry/src/lib.rs:1102-1132).
- The attacker submits from relayer 1320's operational key with slot 1, the 24-bit bitmap, and the aggregate signature.
- `count_ones() = 24` satisfies `MinSignatures`; key collection resolves the 24 selected malicious committee members; `fast_aggregate_verify` succeeds over their genuine signatures (pallets/highway-entry/src/lib.rs:1282-1381).

4. Result.

- `receive_message` records the fresh ID and executes the configured inbound operation (pallets/highway-entry/src/lib.rs:1146-1197).
- A MINT corridor mints the forged amount; a RELEASE corridor transfers it from escrow (pallets/highway-entry/src/lib.rs:2038-2115).
- Because `message_id` is absent from committee selection, the same malicious quorum can sign fresh canonical IDs and repeat for the remainder of the epoch.

Recommended mitigation:

Remove `relayer_id` and freely chosen `slot_number` from committee selection. Derive one protocol-assigned committee from authenticated, unbiased randomness and an objective epoch, slot, or independently fixed source event so submitters cannot retry different seeds. Size the committee and threshold for an explicit lifetime failure

probability; if deterministic safety against every sub-third active-set coalition is required, use an active-set-wide quorum or enforce committee composition instead of random sampling.

Highway: Acknowledged. Caller-controlled grinding remains, but the hardcoded 128/86 parameters and 6,000-relayer cap bound a strict-sub-third coalition to approximately $2.68e-11$ success per epoch across supported set sizes, and to zero below 259 active relayers.

Cyfrin: Rationale accepted with a condition: with `CommitteeSize = 128`, `MinSignatures >= 86`, `MAX_SLOTS <= 5`, and at most 6,000 relayers, a strict-sub-third coalition's full caller-selected grind is bounded to approximately $2.68e-11$ success per epoch ($2.68e-5$ over the modeled 10^6 -epoch lifetime), reaches its maximum at $N = 5,998$, and is impossible below 259 active relayers; recalculate before weakening any bound.

7.3 Low Risk

7.3.1 `Pallet::do_update_active_set` and `verify_bls_attestation` never read the `is_authorized_operator` gate

Description: The registry defines a per-relayer `is_authorized_operator` flag that the protocol treats as the on-chain operator-authorization gate. It is set by the admin-only `authorize_operator` (whose doc comment states the relayer "must be called by admin after registration and before the relayer can be included in the active set"), cleared by `deauthorize_operator`, and cleared cross-pallet by `do_deauthorize_operator` when a relayer-tier NFT is undelegated. The registry even defines a dedicated error `UnauthorizedOperatorInActiveSet` (`pallets/highway-registry/src/lib.rs:597`) documenting the intended invariant that every relayer whose bit is set in an active-set bitmap must be an authorized operator.

`Pallet::do_update_active_set` (`pallets/highway-registry/src/lib.rs:1908-2066`) is the sole writer of active sets, reached by both `update_active_set` and `update_active_set_lossless`. It validates the submitted `active_bitmap` only with `validate_bitmap_size` (size/trailing-bit sanity) and `ensure_registered_subset` (every active bit maps to a registered relayer, `pallets/highway-registry/src/lib.rs:1974`). It never checks `is_authorized_operator` for any active bit. `UnauthorizedOperatorInActiveSet` is defined at `pallets/highway-registry/src/lib.rs:597` and referenced nowhere else - it is dead code. Downstream, `HighwayEntry::verify_bls_attestation` (`pallets/highway-entry/src/lib.rs:1308-1320`) reconstructs the committee purely from the pinned active-set bitmap plus each member's registered BLS key; it also never consults `is_authorized_operator`. The flag is therefore enforced at no point on the inbound critical path - it is only an advisory hint for off-chain bitmap construction.

A direct consequence is that the NFT-undelegation revocation path is a no-op for committee membership: clearing `is_authorized_operator` writes a flag nothing reads, so a relayer that has lost its NFT backing keeps its committee seat and keeps signing inbound attestations until an off-chain updater happens to push a bitmap that drops it - and that update itself re-checks nothing.

Files:

- `Pallet::do_update_active_set`
- `HighwayEntry::verify_bls_attestation`

Impact: The economic-security model that ties committee membership to authorized, NFT-backed operators is not enforced on-chain. A relayer that was never authorized, or was deauthorized after undelegating its backing NFT, can be seated into the signing committee and have its BLS key contribute to threshold attestation. Because inbound `receive_message` mints or releases bridged value on a threshold of committee signatures, seating unauthorized operators dilutes the honest-majority / staked-collateral assumption the bridge rests on. The harm manifests under honest operation: the code documents the invariant (via the dead error and the `authorize_operator` doc comment) and simply never wires the check in, so an operator that legitimately lost authorization remains seatable and stays seated.

Recommended Mitigation: In `do_update_active_set`, after `ensure_registered_subset`, require that every set bit in `active_bitmap` maps to a relayer with `is_authorized_operator == true`, returning the already-defined `Error::<T>::UnauthorizedOperatorInActiveSet` otherwise. The cheapest correct form mirrors `ensure_registered_subset`: maintain an authorized-operator bitmap in lockstep (set on `authorize_operator`, cleared on `deauthorize_operator` / `do_deauthorize_operator` / `remove_relayer`) and require `active & !authorized == 0` byte-wise. As defense in depth, have the deauthorization path also clear the relayer's bit from the current and pending active-set bitmaps so revocation takes effect immediately rather than waiting for the next updater submission.

Highway: Fixed in [64a389d](#), [8869838](#), [3da85c5](#).

Cyfrin: Verified.

7.3.2 `Pallet::remove_relayer` guards only current and pending epochs while the `verify_bls_attestation` pin window keeps the current-1 committee claimable

Description: Inbound `receive_message` uses epoch pinning: a BLS proof is claimable when its pinned epoch equals the current epoch or the epoch immediately before it (`pallets/highway-entry/src/lib.rs:1277-1280`).

When a proof pins to `current_epoch - 1`, the committee is reconstructed from that epoch's retained bitmap, and for every selected member the code reads its live BLS key from the registry; a missing key aborts with `RelayerBlsKeyNotInRegistry` (pallets/highway-entry/src/lib.rs:1341-1342).

`Pallet::remove_relayer` (pallets/highway-registry/src/lib.rs:937-981) is guarded by `is_relayer_in_active_or_pending` (guard at 951) and, on removal, deletes the relayer's `RelayerBlsKeys` entry (964). But that guard inspects only the head set and - when the head is a pending set - the head's immediate predecessor. When no pending set exists, head is the current effective epoch and the guard checks only that epoch's bitmap; it never consults the separate buffer entry for `current_epoch - 1`. A relayer that is in the `current_epoch - 1` set but not in the `current_epoch` set therefore passes the guard and can be removed, deleting its BLS key while an in-flight proof pinned to `current_epoch - 1` still needs it. The guard's epoch coverage (current plus pending) is strictly narrower than the acceptance window the verification path reads (current and current-1).

The proof's deterministic committee is fixed by (epoch randomness, `relayer_id`, `slot`), so no re-attestation can route around the removed relayer for that message id; once the current epoch advances past `current_epoch - 1`, the proof additionally fails the pin-window / epoch-not-found checks. The message is permanently undeliverable while pinned to that epoch.

Files:

- `Pallet::remove_relayer`
- `HighwayEntry::verify_bls_attestation`

Impact: A single honest `remove_relayer` on a relayer that has rotated out of the current committee but is still referenced by a claimable `current_epoch - 1` proof permanently strands that inbound message and its token transfer. No malice is required - the registrar is doing routine cleanup and the guard reports the relayer safe to remove. Because token delivery is atomically coupled to a successful `receive_message` and there is no failed-message retry primitive, and the source chain has already debited, the bridged funds strand pending source-side admin recovery. The impact is a per-message denial of delivery, triggered by honest privileged operation.

Recommended Mitigation: Widen the removal guard so it rejects removing any relayer whose bit is set in any retained active set whose epoch is $\geq \text{current_epoch.saturating_sub}(1)$ - i.e. mirror the exact set of epochs the inbound verification path will accept. Alternatively, retain a removed relayer's `RelayerBlsKeys` entry (tombstoned) until every epoch that references it has aged out of the pin window, decoupling registry removal from key availability.

Highway: Fixed in [64a389d](#), [88730a7](#).

Cyfrin: Verified.

7.3.3 `HighwayConfig::register_token` stores an admin-supplied `local_decimals` never reconciled against the asset's `pallet_assets` metadata decimals

Description: `HighwayConfig::register_token` (pallets/highway-config/src/lib.rs:525-564) accepts a `local_decimals: u8` parameter from the admin and copies it verbatim into the stored `TokenInfo` (pallets/highway-config/src/lib.rs:531,550). It performs no cross-check against the asset's own on-chain decimals, which are recorded in `pallet_assets::Metadata(asset_id).decimals` and are readable on-chain. The stored `local_decimals` is the value that drives inbound amount conversion: `resolve_inbound_transfer` passes `token_info.local_decimals` into `convert_amount_decimals` (pallets/highway-entry/src/lib.rs:1676-1695), which computes the local amount actually minted or released for the corridor's backing math.

A mismatch between the admin-supplied `local_decimals` and the asset's real metadata decimals therefore silently rescales every inbound transfer for that corridor by a power of ten - the difference between the two decimal values. Unlike the source-chain decimals, whose correctness the protocol treats as governance-checked at corridor registration with no on-chain verification against the real source-chain token, the local decimals correspond to an asset that lives on this chain and whose true decimals are on-chain-verifiable, so this value can be reconciled in code but is not.

Files:

- `HighwayConfig::register_token`

- `HighwayEntry::convert_amount_decimals`

Impact: If `local_decimals` is configured wrong for a corridor, every inbound mint or release on that corridor is scaled by a power of ten relative to the intended amount - over-crediting the recipient when the stored value is too large, or short-paying when it is too small, against the asset's real precision. Because the value feeds the mint/release backing math for all inbound messages on that corridor, the error is systematic rather than per-message. The harm manifests under honest admin operation: a single mistyped or stale decimals value at registration is silently accepted and mis-scales the corridor thereafter, with no on-chain guard catching the divergence.

Recommended Mitigation: In `register_token`, read the asset's recorded decimals from its `pallet_assets` metadata and reject registration when the supplied `local_decimals` does not match, rather than trusting the caller-supplied value. If the metadata may legitimately be unset at registration time, require it to be set first (or defer the decimals capture to a point where the metadata is present) so the stored `local_decimals` is always reconciled against the asset's true on-chain precision.

Highway: Fixed in [3e4862b](#).

Cyfrin: Verified.

7.3.4 `HighwayEntry::execute_inbound_native_operation` discards the deposit-creating imbalance on a native MINT

Description: On the native-token inbound MINT branch, `HighwayEntry::execute_inbound_native_operation` credits the recipient with `let _ = T::NativeCurrency::deposit_creating(recipient, native_amount);` (`pallets/highway-entry/src/lib.rs:2042`) and discards the returned `PositiveImbalance`. `NativeCurrency` resolves to `Balances`. The `pallet_balances` implementation of `deposit_creating` runs its credit inside a `try_mutate_account_handling_dust(...)` guarded by `ensure!(value >= ed || !is_new, ...)` and terminates with `.unwrap_or_else(|_| PositiveImbalance::zero())` (verified against the resolved `pallet-balances` source `impl_currency.rs`, out of scope, traced for context). When the recipient account does not yet exist and `native_amount` is below the existential deposit, the credit errors internally, the outer `unwrap_or_else` swallows it and returns a zero imbalance: nothing is credited and no error propagates.

`receive_message` inserts the replay marker `ExecutedInboundMessages::insert(message_id, ())` at `pallets/highway-entry/src/lib.rs:1146` before the token operation, so the message is permanently marked executed. The MINT branch then emits `TokenMinted` and returns `Ok()`, committing the extrinsic. Every sibling inbound value path propagates its error and reverts instead: asset MINT uses `mint_into(...).map_err(|_| MintFailed)?` (`pallets/highway-entry/src/lib.rs:2095`), and both native and asset RELEASE use `transfer(...).map_err(|_| ReleaseFailed)?` (`pallets/highway-entry/src/lib.rs:2055-2061`). The native MINT branch is the only one that discards its result.

Files:

- `HighwayEntry::execute_inbound_native_operation`
- `HighwayEntry::receive_message`

Impact: Silent, permanent loss of bridged funds for any below-existential-deposit native MINT delivery to a fresh (or previously-reaped) recipient. Nothing reverts, so the source chain treats the transfer as complete; the protocol spec commits that the runtime reverts inbound messages on any amount-derived failure and leaves the source-chain funds debited pending admin recovery (per `README.md`), but here the message succeeds, the replay slot is consumed, a misleading `TokenMinted` event fires, and the recipient receives nothing. The recipient is chosen by the source-chain sender, so this hits ordinary users. Per-event magnitude is bounded by the existential deposit (one `MILLI_UNIT`), and recovery is not available, hence Low.

Recommended Mitigation: Do not discard the mint result. Either use the fallible `fungible::Mutate::mint_into` API and `?-propagate` its error (mirroring the asset path), or inspect the returned imbalance and `ensure!(credited == native_amount, Error::<T>::MintFailed)` so the extrinsic reverts and the message stays retryable. Do not rely solely on a config-time `min_amount >= ExistentialDeposit` floor, because the reaped-recipient case still loses funds; the runtime credit check is required.

Highway: Fixed in [677d77f](#), [9f0980e](#).

Cyfrin: Verified.

7.3.5 `NftDelegationRegistry::complete_undelegation` is permissionless and lets a bystander force-complete a pending undelegation

Description: `NftDelegationRegistry::complete_undelegation` is `ensure_signed` and discards the caller identity with no owner check (`pallets/highway-nft-delegation-registry/src/lib.rs:474-521`) - anyone can complete a pending undelegation once the cooldown guard `current_block >= completion_block` passes. The deployed runtime ships `DefaultUndelegationCooldown = 0`, so `undelegate` sets `completion_block = current_block.saturating_add(0) = current_block` and the completion guard passes in the same block the undelegation is initiated. Combined, any signer can drive a pending undelegation to completion the instant it is initiated: Delegations is taken, the NFT is unbound, and for a relayer-tier (Path A) NFT the operator-backing slot `RelayerOperatorTier` is cleared and `deauthorize_operator` is called.

Because completion frees the operator-backing slot with no delay, a relayer's Path-A backing slot can be reopened and re-seized within a single block. Path-A self-bind is itself permissionless as to which relayer is backed - any relayer-tier NFT holder can occupy an unbacked registered relayer's slot - so the zero-delay, bystander-completable teardown tightens the window in which a slot can be flipped from one occupant to another.

Files:

- `NftDelegationRegistry::complete_undelegation`

Impact: A bystander can force-complete an operator's pending undelegation immediately, removing the operator's control over the timing of their own teardown and, under the zero cooldown, collapsing the anti-slash lock window the pallet documents. The affected NFT is unbound and the operator-backing slot is reopened without the operator's consent to the completion timing. This is a permissionless grieving/race-tightening on delegation-lifecycle state, not a direct theft of third-party funds, hence Low.

Recommended Mitigation: Restrict `complete_undelegation` to the recorded delegator (`ensure!(who == delegation.delegator, ...)`), matching the ownership check `undelegate` already performs, so only the initiating party can complete their own pending undelegation. Independently, ship a non-zero `DefaultUndelegationCooldown` so completion cannot occur in the same block the undelegation is initiated.

Highway: Acknowledged. [82d7ebb](#) fixes permissionless completion. The zero-cooldown half is accepted for the MVP because slashing is not operationalized and operator misconduct is handled by NFT revocation; a cooldown will be introduced with slashing for the operator backing NFT.

Cyfrin: Rationale accepted with a condition: [82d7ebb](#) removes the bystander-completion vector; the remaining zero-cooldown teardown is acceptable for the MVP only while `slash_delegation` remains unused, and a slash-safe cooldown or completion gate must be enforced before slashing is relied upon.

7.3.6 Deployed `CommitteeSize=34` and `MinSignatures=24` violate the pallet `integrity_test` floor `MinSignatures>=87`

Description: The runtime wires `CommitteeSize = 34` and `MinSignatures = 24` (`runtime/src/configs/mod.rs:344-345`), a two-thirds-plus-one threshold of 34 per the in-code comment. `HighwayEntry::integrity_test` asserts `T::MinSignatures::get() >= 87` and `T::CommitteeSize::get() <= 128` (`pallets/highway-entry/src/lib.rs:741-761`), because `receive_message` was benchmarked over a `Linear<87,128>` signer domain and the asserts exist to trip when the constants drift outside the measured range. `24 >= 87` is false, so the first assert fails wherever `integrity_test` runs (unit tests, try-runtime release gating, benchmark genesis). `integrity_test` is not on the block-production path, so a live chain is not halted, but the guard is permanently tripped and can no longer catch a genuine out-of-range drift. Separately, `receive_message`'s generated weight was fitted for signer counts in `[87,128]` while the deployed committee signs with counts in `[24,34]`, so the charged weight is evaluated below its measured domain (the two sub-facets of that weight defect are detailed in the clustered weight finding covering the `Linear` domain and the active-set reconstruction loop).

Files:

- `HighwayEntry::integrity_test`

Impact: No fund loss and no live-chain halt. Two concrete current defects: the pallet's own integrity guard rejects the shipped configuration (so any test or `try-runtime` integrity check panics and the guard's drift-detection value is lost), and inbound-message weight is charged from a model measured for a committee the chain never reaches. If the team's intent was the 128-seat committee referenced elsewhere in the design, then 34/24 is a materially weaker committee threshold than documented; the authoritative value should be confirmed. Recoverable by reconciling the three sources (config, asserts, benchmark), hence Low.

Recommended Mitigation: Resolve intent and make the config, the `integrity_test` bounds, and the benchmark domain agree. If 34/24 is intended, re-run the `receive_message` benchmark with a `Linear<24,34>` domain, update the benchmark bound, and relax the asserts in `HighwayEntry::integrity_test` to the new floor (as the assert's own comment instructs). If 87/128 is intended, raise `CommitteeSize/MinSignatures` at `runtime/src/configs/mod.rs:344-345`. Then propagate the resolved values to the bootstrap scripts and receive tooling.

Highway: Fixed in [c66a229](#).

Cyfrin: Verified.

7.3.7 `NftDelegationRegistry::slash_delegation` lowers `RelayerDelegatedWeight` but never the `RelayerOperatorTier` capacity ceiling

Description: `NftDelegationRegistry::slash_delegation` reduces the delegation's `internal_value` and decrements `RelayerDelegatedWeight` by the slash amount, and for a Path-A operator NFT persists the reduced value to the NFT's `NominalValue` (`pallets/highway-nft-delegation-registry/src/lib.rs:568-631`). But the relayer's delegation-capacity ceiling is derived from the tier enum stored in `RelayerOperatorTier` (`max_delegation_capacity = tier.internal_value() * 2`, `pallets/highway-nft-delegation-registry/src/lib.rs:380`), which slashing never mutates, and `slash_delegation` never calls `deauthorize_operator` even when the slash drives the backing to zero. The `deauthorize_operator` and `RelayerOperatorTier::remove` teardown only runs in `complete_undelegation` (`pallets/highway-nft-delegation-registry/src/lib.rs:510-513`).

Consequently, after an operator's backing NFT is slashed toward zero, the relayer's capacity ceiling stays at the full tier constant while its own contribution to `RelayerDelegatedWeight` drops - so the delegator headroom (`max_capacity - RelayerDelegatedWeight`) actually increases by the slashed amount, and the relayer remains `is_authorized_operator == true`. The governance action whose purpose is to reduce a misbehaving relayer's standing instead opens more delegation headroom under it and leaves it authorized.

Files:

- `NftDelegationRegistry::slash_delegation`
- `NftDelegationRegistry::complete_undelegation`

Impact: The delegated-weight-versus-operator-stake collateral relationship is not enforced against a slashed operator: a slashed-to-zero relayer keeps its full tier ceiling, can attract up to the full tier's worth of delegator weight, and stays authorized. Because `internal_value` is an off-chain reward-share weight rather than an on-chain token balance, no direct token movement occurs; the impact is broken economic-backing accounting and mis-weighted off-chain reward input. Recoverable by governance separately removing the tier / `deauthorizing`, hence Low.

Recommended Mitigation: In `slash_delegation`, when the slashed NFT is a relayer-tier (Path A) backing NFT and the slash drives the backing to (or below) a configured floor, either call `deauthorize_operator` and clear `RelayerOperatorTier` (mirroring `complete_undelegation`'s relayer-tier branch), or derive `max_capacity` from the operator's live post-slash nominal value rather than the frozen tier constant so headroom shrinks with the slash.

Highway: Acknowledged. `slash_delegation` is a dormant governance capability, not the MVP punishment path: a misbehaving operator is removed via `remove_relayer + clear_relayer_delegations`, which clear `RelayerOperatorTier` and `deauthorize`, so the reopened-capacity / still-authorized state never arises as deployed. If on-chain slashing later becomes an active economic path, the operator self-bind weight will be tracked separately and, on backing reaching zero, `RelayerOperatorTier` cleared and `deauthorize_operator` called.

Cyfrin: Rationale accepted with a condition: `slash_delegation` must remain unused and operator punishment must use the full removal-and-delegation-cleanup workflow; before on-chain slashing is relied upon, zero backing must clear `RelayerOperatorTier` and `deauthorize` the operator in the slash path.

7.3.8 NftDelegationRegistry::complete_undelegation Path A undelegation orphans the Path B delegators

Description: When a relayer-tier (Path A) operator NFT completes undelegation, `NftDelegationRegistry::complete_undelegation` removes `RelayerOperatorTier` and calls `deauthorize_operator` (`pallets/highway-nft-delegation-registry/src/lib.rs:510-513`), but does not touch the Path B delegator NFTs still recorded in `RelayerDelegations/Delegations`, still contributing to `RelayerDelegatedWeight`, and still bound (transfer-locked) to that relayer. After the operator exits, the relayer has no operator tier: `get_remaining_capacity` returns `None` and any fresh Path B delegate fails `RelayerNotBacked`, yet the pre-existing delegator NFTs remain locked and delegated to a now-deauthorized, unbacked relayer. No event marks the orphaning, so delegators receive no signal, and recovery is manual per-delegator via their own `undelegate/complete_undelegation` (`pallets/highway-nft-delegation-registry/src/lib.rs:335-362`).

Files:

- `NftDelegationRegistry::complete_undelegation`
- `NftDelegationRegistry::delegate`

Impact: Path B delegators are left with NFTs locked and productively idle under a relayer that has lost its backing and authorization, with no notice. They can self-recover by individually calling `undelegate/complete_undelegation`, so this is temporary loss-of-use and lost reward opportunity rather than a permanent lock. Hence Low.

Recommended Mitigation: When a Path A operator NFT completes undelegation while Path B delegations remain, either (a) block operator undelegation until all Path B delegations are cleared, (b) auto-cancel and auto-unbind the dependent Path B delegations (as `clear_relayer_delegations` does), or (c) emit an explicit orphaning event and document that delegators must re-home. Do not silently leave delegator NFTs bound under an unbacked, deauthorized relayer.

Highway: Acknowledged, as-designed. Path-B delegators are never locked and can self-recover at any time (`undelegate / complete_undelegation` have no backing requirement and zero cooldown); once the operator exits, new Path-B delegates fail `RelayerNotBacked` and the orphaned weight is inert (relayer deauthorized, in no committee). The operator is deliberately allowed to exit rather than be trapped or cascade-unbind third-party NFTs; `NftUndelegated` is emitted and the unbacked state is observable, so affected delegators are detected and prompted to re-delegate off-chain.

Cyfrin: Rationale accepted with a condition: Path-B NFTs remain transfer-locked until their owners exit, but a zero-block undelegation cooldown plus reliable off-chain detection and notification bounds the impact to temporary loss of use and reward opportunity.

7.3.9 HighwayEntry::execute_inbound_native_operation uses AllowDeath on the native RELEASE from a vault shared with asset corridors

Description: The native inbound RELEASE branch transfers from the escrow account to the recipient with `ExistenceRequirement::AllowDeath` (`pallets/highway-entry/src/lib.rs:2055-2061`), unlike the native fee transfer which uses `KeepAlive` (`pallets/highway-entry/src/lib.rs:2277-2296`). When a release drains the pooled escrow account's native balance to a non-zero remainder below the existential deposit, `AllowDeath` lets the escrow account be reaped and the sub-ED remainder is burned via dust removal. The deployment scripts point the native, USDT, and ETH corridors at one shared escrow account (deployment support, out of scope, traced for context), so reaping the escrow's native balance drops the provider reference that keeps its co-located `pallet_assets` escrow entries alive, which can strand the asset escrow backing.

Files:

- `HighwayEntry::execute_inbound_native_operation`

Impact: A native RELEASE that straddles the existential-deposit boundary on the shared escrow account can reap it and burn the sub-ED remainder, and because the same account also holds asset-corridor escrow, losing its provider reference can affect the co-located asset escrow. Per-event native loss is dust-level, and the escrow can be re-bootstrapped above the existential deposit, so this is a recoverable accounting/liveness leak rather than a large direct loss, hence Low.

Recommended Mitigation: Use `ExistenceRequirement::KeepAlive` on the native RELEASE transfer (matching the native fee path) so a release can never reap the escrow account, or maintain a per-corridor escrow account rather than a single shared account so a native drain cannot affect co-located asset escrow. Reconcile the shared-escrow deployment configuration with whichever choice is made.

Highway: Fixed in [d4db342](#), [a256ee0](#).

Cyfrin: Verified.

7.3.10 `HighwayConfig::configure_native_token_for_chain` accepts a `min_amount` below the existential deposit when `native_decimals <= source_decimals`

Description: `HighwayConfig::configure_native_token_for_chain` validates a corridor's minimum amount through `validate_decimal_minimum`, which early-returns `Ok` for `local_decimals <= remote_decimals` (`pallets/highway-config/src/lib.rs:1224-1226`), and `validate_amount_range`, which floors `min_amount` only at `> 0` (`pallets/highway-config/src/lib.rs:1177-1180`). Neither checks `min_amount` against the runtime existential deposit. Native decimals are 18; for a source representation with at least 18 decimals the decimal-floor branch is a no-op, so an admin can validly set `min_amount` far below the existential deposit (`MILLI_UNIT`). An inbound native MINT of an amount in `[min_amount, ExistentialDeposit)` then reaches the native MINT branch, whose discarded `deposit_creating` credits nothing for a fresh recipient and returns success (`pallets/highway-entry/src/lib.rs:2038-2052`) - a silent-loss no-op.

Files:

- `HighwayConfig::configure_native_token_for_chain`
- `HighwayEntry::execute_inbound_native_operation`

Impact: A corridor configured through the documented flow can accept inbound native MINT amounts below the existential deposit, which then silently evaporate on delivery to a fresh recipient while the message is marked executed. The config surface offers no way to guarantee `min_amount >= ExistentialDeposit`, so the misconfiguration is not obvious at configuration time and there is no recovery once a below-ED delivery no-ops. The configuration itself is admin-correctable, but the value loss on an affected delivery is not recoverable; per-event magnitude is bounded by the existential deposit, hence Low.

Recommended Mitigation: In `HighwayConfig::configure_native_token_for_chain` (and the shared `validate_decimal_minimum/validate_amount_range` path), floor `min_amount` at the native existential deposit for native MINT-inbound corridors - ensure! `(min_amount >= <T::NativeCurrency>::minimum_balance(), ...)` - so a corridor can never accept an amount that would convert below the existential deposit. This must accompany, not replace, the runtime credit check on the mint path, since a recipient live at signing time can be reaped before claim time.

Highway: Fixed in [677d77f](#), [9f0980e](#).

Cyfrin: Verified.

7.3.11 `Pallet::select_committee` orders active_relayers ascending by id and depends on the off-chain builder agreeing on that ordering

Description: Inbound `receive_message` reconstructs the signing committee on-chain and then verifies the relayers' aggregate BLS signature against that reconstruction. The committee is drawn by hypergeometric rejection sampling: for each seat, `Pallet::select_committee` hashes the epoch randomness with the seat and retry counters, reduces the hash modulo the active-relayer count to a position candidate, and resolves that position to a relayer id via `active_relayers[candidate]` (`pallets/highway-entry/src/lib.rs:1425-1445`). The `active_relayers` slice it indexes is built by iterating relayer ids `1..=max_relayer_id` in ascending order and keeping those whose bit is set in the epoch's active-set bitmap (`pallets/highway-registry/src/lib.rs:2551-2553`), where the bit for a relayer sits at position `id - 1` (`Pallet::check_bit_in_bitmap`, `pallets/highway-registry/src/lib.rs:1839`). The committee that the counterpart chain's off-chain builder selects must produce the byte-identical position-to-id mapping and the same `max_relayer_id` semantics, or the pubkeys the relayers actually sign with will not match the pubkeys the pallet reads for each seat, and `fast_aggregate_verify` will not succeed.

That cross-chain agreement is load-bearing but is asserted only by an in-code comment ("must match Ethereum exactly", `pallets/highway-entry/src/lib.rs:1400`), not by any shared, versioned specification. The two sides encode the mapping in independent codebases (the counterpart builder is out of scope, traced for context), so a divergence in id ordering, the `id - 1` bit convention, or how each side derives `max_relayer_id` silently breaks reconstruction.

Files:

- `Pallet::select_committee`
- `Pallet::check_bit_in_bitmap`

Impact: If the off-chain committee builder and the on-chain `select_committee` disagree on the position-to-id mapping or on `max_relayer_id`, the reconstructed committee membership differs from the set that produced the signatures, so BLS verification fails for the affected inbound messages and those messages cannot be claimed. This is a liveness/correctness risk on the inbound bridge path (delivery fails rather than mis-pays), and it is a structural coupling with no on-chain guard: nothing enforces the two implementations stay in sync, so any future edit to id ordering or bitmap sizing on either side can reintroduce the divergence.

Recommended Mitigation: Promote the position-to-id mapping and `max_relayer_id` derivation from an in-code comment to a shared, versioned specification that both this runtime and the off-chain committee builder implement against, and add cross-implementation test vectors (fixed epoch randomness, active-set bitmap, and expected committee id list) exercised in CI on both sides so a drift in the ascending-id ordering or the `id - 1` bit convention fails a test rather than silently breaking inbound BLS verification.

Highway: Acknowledged; The canonical ascending-ID mapping is documented and tested on the Substrate side; the team considers the EVM implementation symmetrical and leaves the portable vector available for a future counterpart test.

Cyfrin: Rationale accepted with a condition: every EVM and off-chain committee implementation must follow the shared ascending-ID, `id - 1` little-endian bitmap, and snapshot `max_relayer_id` rules and run the portable vector in CI so ordering drift cannot silently break inbound BLS verification.

7.3.12 Bootstrap scripts register only 16 relayers while the deployed runtime requires `CommitteeSize=34`

Description: The bootstrap flow registers only 16 relayers and marks all 16 active (the relayer-registration script hardcodes 16 and sets `includeAllRelayers: true`; deployment support, out of scope, traced for context), and the receive tooling builds attestations for a 16-seat / 11-signature committee. But `HighwayEntry::verify_bls_attestation` requires the active set to be at least the committee size, reverting `ActiveRelayerSetTooSmall` when `n_active` is below `CommitteeSize` (`pallets/highway-entry/src/lib.rs:1313`), and the deployed runtime sets `CommitteeSize = 34` (`runtime/src/configs/mod.rs:344`). With 16 active relayers and a 16-seat proof, the active-set-size check fails before any signature threshold is even reached, so no inbound message can complete when bootstrapping from these scripts.

Files:

- `HighwayEntry::verify_bls_attestation`

Impact: The documented dev/testnet bootstrap yields a non-functional inbound bridge: every cross-chain receive reverts deterministically because the active set is smaller than the required committee size. This is a deployment-configuration inconsistency between the bootstrap artifacts (16 relayers) and the runtime committee parameter (34). The operator hits a self-evident revert on the first receive rather than any silent loss, and the mismatch is fixed by re-sizing the bootstrap, hence Low.

Recommended Mitigation: After reconciling the authoritative committee geometry, size the bootstrap to match: register at least `CommitteeSize` active relayers (ideally two times the committee size per the pallet's own guidance), and update the receive tooling's committee-size / signature constants and the relayer-count constants in the registration and delegation-setup scripts to the real runtime values.

Highway: Fixed in [01a3c9e](#), [c882548](#).

Cyfrin: Verified.

7.3.13 BLS committee private keys and libp2p private keys are committed to the repository

Description: The relay bootstrap material - 16 relay entries each carrying a plaintext BLS12-381 private signing key, plus per-relayer libp2p ed25519 private keys - is committed to version control (deployment support, out of scope, traced for context). The keys are additionally fully deterministic from hardcoded public per-relayer development seed strings for the BLS and libp2p keys, and the ignore rules exclude environment and dependency files but not the key files, so any repository reader can reconstruct every committee member's signing key. These are the keys the inbound path's aggregate verification checks against.

Files:

- `HighwayEntry::verify_bls_attestation`

Impact: On a throwaway dev chain the keys are disposable, so direct impact is low. The risk is reuse: if the deterministic dev seeds or committed keys are ever registered on a staging or production relay set without rotation, any repository reader controls the entire committee and can forge aggregate attestations to satisfy inbound verification for arbitrary messages. Because exploitation requires reuse on a value-bearing chain rather than an on-chain trigger, this is Low - but committed, derivable committee private keys should never sit in the tracked tree.

Recommended Mitigation: Remove the committed relay key file and per-relayer libp2p key files from version control and add them to the ignore rules; treat bootstrap key output as local-only artifacts. For any non-dev deployment, generate relay BLS and libp2p keys from high-entropy secrets held in a secret manager - never from the deterministic seed strings - and never register the committed dev keys on a chain holding real value.

Highway: Fixed in [21a5929](#).

Cyfrin: Verified.

7.3.14 `HighwayEntry::verify_bls_attestation` enforces no upper bound on the relay-supplied `ttl`

Description: `HighwayEntry::verify_bls_attestation` checks the relay-supplied `ttl` only against a lower bound (`ttl >= current_block`, `pallets/highway-entry/src/lib.rs:1262`), with no upper bound. Committee membership is pinned by epoch and the claim window admits the current and previous epoch, a design that relies on a proof's TTL being far shorter than an epoch so a proof cannot outlive the committee that signed it. But `current_epoch` is data-driven - it advances only when an active-set rotation lands, not on a block clock - so during a chain or rotation stall a proof carrying a large `ttl` stays simultaneously TTL-valid and pin-window-valid indefinitely (`pallets/highway-entry/src/lib.rs:1260-1280`). The property that bounds how long a proof can be claimed therefore degrades from a hard TTL bound to an assumption that rotations keep advancing.

Files:

- `HighwayEntry::verify_bls_attestation`

Impact: With no `ttl` ceiling, a relay can attach an arbitrarily long TTL, and if epoch rotation stalls the proof remains claimable for the whole stall rather than expiring on schedule. This weakens the intended freshness bound on in-flight proofs to a rotation-liveness assumption. No direct fund loss follows from the missing bound on its own; it is a hardening gap in the freshness model, hence Low.

Recommended Mitigation: Enforce an upper bound on `ttl` in `verify_bls_attestation` - introduce a maximum-proof-TTL config constant and require that `ttl` not exceed `current_block` plus that configured maximum - so a proof's claimable lifetime is hard-bounded independently of whether epoch rotation continues to advance.

Highway: Acknowledged as a defense-in-depth gap. `ttl` is bound into the committee-signed BLS preimage, so it is not an attacker-controlled value; an overlong TTL would require committee-level collusion or faulty relay software combined with a rotation stall, which falls outside the uncompromised-network threat model. The proposed on-chain `ttl` bound is therefore not added.

Cyfrin: Rationale accepted with a condition: every honest committee signer must independently derive and reject TTLs beyond the documented short horizon from a fresh destination head; otherwise a quorum-signed overlong proof remains claimable throughout a frozen-epoch rotation stall.

7.3.15 `Pallet::receive_message` **folds** `payload_target_address` **into the message-id but discards it during execution**

Description: On inbound, `payload_target_address` is `.encode()`-ed and folded into the message-id preimage and BLS attestation (`pallets/highway-entry/src/lib.rs:1061,1106-1109`), so the committee attests to it. But at execution the payload is handled only by extracting the leading pallet/call index bytes, checking them against the whitelist, and calling `execute_payload`, which decodes the payload as a `RuntimeCall` and dispatches it with `RawOrigin::Signed(sender)` where `sender` is the relayer who delivered the message (`pallets/highway-entry/src/lib.rs:1199-1218,1798-1811`). The attested `payload_target_address` is never read to select a target, origin, or recipient - it is committed into the message id but semantically dropped at dispatch, and the effective actor is always the delivering relayer. This is latent under the current whitelist, which only admits `System::remark`.

Files:

- `HighwayEntry::receive_message`
- `HighwayEntry::execute_payload`

Impact: The message-designated payload target is authenticated by the committee but not honored: every payload runs under the relayer's own signed origin rather than the attested target. Today the whitelist admits only an inert call, so no privilege or value is reachable through this path, which is why the impact is latent and Low. The concern is structural - if a future whitelist entry grants anything meaningful under a signed origin, the executing principal would be the arbitrary delivering relayer rather than the attested target.

Recommended Mitigation: Consume the attested `payload_target_address` at dispatch: derive the execution origin from the attested target (rather than from the relayer) so the payload runs as the principal the committee attested, or, if the relayer origin is intended, drop `payload_target_address` from the preimage so the committee does not appear to authenticate a target the execution ignores. Keep the whitelist minimal until the target-versus-origin semantics are resolved.

Highway: Acknowledged. `payload_target_address` is authenticated metadata; the signed `RuntimeCall` carries its own destination, and payload authority comes from the dedicated pallet account rather than this field.

Cyfrin: Rationale accepted with a condition: Substrate integration documentation and every cross-chain message builder must treat `payload_target_address` as authenticated metadata only - the committee-authenticated canonical `RuntimeCall` determines the executed call and any destination arguments, while the dedicated keyless payload account is the sole dispatch origin.

7.3.16 `NftDelegationRegistry::clear_relayer_delegations` **removes** `RelayerOperatorTier` **but never calls** `deauthorize_operator`

Description: `NftDelegationRegistry::clear_relayer_delegations` tears down a relayer's delegation state and removes `RelayerOperatorTier` (`pallets/highway-nft-delegation-registry/src/lib.rs:653-685`), but omits the `deauthorize_operator` call that its sibling `complete_undelegation` makes on the relayer-tier teardown path (`pallets/highway-nft-delegation-registry/src/lib.rs:510-513`). The two teardown paths that both remove `RelayerOperatorTier` are asymmetric: `complete_undelegation` deauthorizes, `clear_relayer_delegations` does not. If an admin uses `clear_relayer_delegations` to remove a relayer's NFT backing without a subsequent `deauthorize_operator` or `remove_relayer`, the relayer's `is_authorized_operator` flag stays true with no NFT backing it. The desync is bounded today because the `is_authorized_operator` flag is not read on the on-chain critical path - `do_update_active_set` (`pallets/highway-registry/src/lib.rs:1908-2066`) enforces registered-subset but never checks operator authorization - so the stale flag has no current consumer.

Files:

- `NftDelegationRegistry::clear_relayer_delegations`
- `NftDelegationRegistry::complete_undelegation`

Impact: Asymmetric teardown leaves a latent authorization desync: a relayer stripped of its backing via `clear_relayer_delegations` remains flagged as an authorized operator. The flag is not consulted on-chain today, so there is no active exploit, but the inconsistency is a drift hazard - if a future consumer starts enforcing `is_`

authorized_operator, an unbacked relayer torn down through this path would be treated as authorized. Latent and admin-triggered, hence Low.

Recommended Mitigation: Make the two teardown paths symmetric: have `clear_relayer_delegations` call `deauthorize_operator(relayer_id)` when it removes `RelayerOperatorTier`, exactly as `complete_undelegation` does on the relayer-tier branch, so a relayer's authorization flag is always cleared together with its backing tier.

Highway: Fixed in [6b7ff88](#).

Cyfrin: Verified.

7.3.17 A relayer operator's own Path-A NFT weight counts against the relayer's delegation capacity ceiling

Description: Path A `NftDelegationRegistry::delegate` adds the operator's own backing NFT `internal_value` into the same `RelayerDelegatedWeight` accumulator that the capacity check compares against `max_delegation_capacity = internal_value * 2` (`pallets/highway-nft-delegation-registry/src/lib.rs:377-419`). The tier documentation describes the ceiling as external-delegation capacity - a relayer "can accept up to twice their own NFT's internal value ... in delegated NFTs" (`pallets/highway-nft-types/src/lib.rs:89-104`). Because the operator's own weight is inside the running total that the ceiling bounds, the operator's self-bind consumes half the ceiling: a Nomad operator's own 70,000 consumes half of the 140,000 ceiling, leaving only 70,000 for external delegators rather than the full documented 140,000.

Files:

- `NftDelegationRegistry::delegate`

Impact: The usable external-delegation capacity is half the documented tier maximum, because the operator's own bond counts against the same ceiling. Whether this is a defect or intended depends on the spec reading of the ceiling, but as written the code and the tier documentation disagree on what the ceiling bounds. No fund movement results (delegated weight feeds off-chain reward accounting), so the impact is a capacity-accounting and documentation inconsistency, hence Low.

Recommended Mitigation: Reconcile the ceiling semantics with the documented intent. If the ceiling is meant to bound external-delegator weight, check `new_weight` against the ceiling excluding the operator's own Path-A contribution (track operator-self weight separately), or raise the ceiling to account for the self-bind. If the current behavior is intended, correct the tier documentation in `pallet-highway-nft-types` to state that the operator's own weight counts against the ceiling.

Highway: Fixed in [da37c07](#).

Cyfrin: Verified.

7.3.18 `highway_config` implements a single mutable Admin while `scripts/README.md` asserts an immutable max-2-admin set

Description: The protocol documentation states that the admin set is fixed at genesis with a maximum of two admins and no runtime extrinsic that mutates it (per `scripts/README.md`). The implementation diverges on every count: Admin is a single `StorageValue` holding one `AccountId`, not a bounded two-element set (`pallets/highway-config/src/lib.rs:222-224`), and `update_admin` rotates that single value at runtime (`pallets/highway-config/src/lib.rs:1127-1138`). Every admin-gated capability across the Highway pallets, and the cross-pallet `EnsureHighwayAdmin` adapter, reads this one mutable value. There is no second admin and no immutability, so the deployed model has no admin redundancy and a single-key compromise is total control.

Files:

- `HighwayConfig::update_admin`

Impact: The deployed admin model contradicts the documented max-two-admin, runtime-immutable commitment. An integrator or off-chain monitor built on the documented model - for example one that watches only for a genesis admin change and never for a runtime `AdminUpdated` event - is wrong about how control can move, and the

intended two-admin redundancy does not exist, so there is no second admin to fall back on if the single key is lost or compromised. No direct exploit follows from the divergence itself; it is a documentation/architecture mismatch with a security-relevant consequence (no redundancy), hence Low.

Recommended Mitigation: Reconcile code and documentation. Either implement the documented model - a bounded two-element admin set with no runtime mutator - or correct scripts/README.md to describe the actual model (a single admin, rotatable at runtime via `update_admin`). If runtime rotation is retained, add a two-step propose/accept rotation so `update_admin` cannot brick admin control by pointing at an unsignable account.

Highway: Fixed in [7990a69](#).

Cyfrin: Verified.

7.3.19 `HighwayEntry::receive_message` weight is benchmarked over a `Linear<87,128>` signer domain that diverges from the deployed 34-member committee

Description: `HighwayEntry::receive_message` charges `T::WeightInfo::receive_message(relayer_bitmap.count_ones())` - a weight modeled solely as a function of the signer count. Two independent dimensions of that model diverge from what the deployed runtime actually executes, so the inbound hot path is mis-priced in two distinct ways. The specifics are enumerated below.

1. **`WeightInfo::receive_message` signer-count domain.** The generated weight is a linear model fitted over signer counts in `[87,128]` (`pallets/highway-entry/src/weights.rs:149-150`), the 128-seat mock committee, while the deployed runtime configures `CommitteeSize = 34 / MinSignatures = 24` (`runtime/src/configs/mod.rs:344-345`), so the production bitmap `count_ones` lands in `[24,34]`. The charged weight is therefore an extrapolation of the fitted line below its measured range rather than a measurement of the deployed signer count.

Recommended: Re-run the benchmark with the deployed `CommitteeSize=34 / MinSignatures=24` constants and update the `Linear<..>` bounds accordingly.

2. **Active-set reconstruction loop dimension.** The active-set reconstruction the inbound path performs on every message iterates `1..=max_relayer_id` (the committee reconstruction in `HighwayEntry::select_committee` and the registry's active-set reconstruction at `pallets/highway-registry/src/lib.rs:1835-1846`), so its cost scales with `max_relayer_id`, which ratchets monotonically toward the 6000 cap as the registry fills and is never decremented on removal. The benchmark pins the active-set size to a small fixed value (committee size times two), so this cost is baked into the constant intercept and no benchmark component scales with it - the weight does not track the true iteration cost as `max_relayer_id` grows.

Recommended: Dimension the benchmark component by the active-set reconstruction bound actually iterated (the active-set / `max_relayer_id` size), not a fixed committee size, and charge that dimension in the dispatch weight closure.

Files:

- `HighwayEntry::receive_message`
- `WeightInfo::receive_message`
- `HighwayEntry::select_committee`

Impact: The inbound bridge hot path is priced by a weight that does not reflect executed cost. The signer-domain facet over-charges relative to the deployed committee (the fitted line is evaluated below its measured range), while the missing active-set-size dimension under-charges the reconstruction loop, and that under-count grows automatically with `max_relayer_id` as the registry fills toward its supported capacity rather than being a fixed offset a single re-benchmark corrects. The under-count means the block-resource reservation for inbound messages is too low, so at scale more inbound messages can be admitted per block than their true cost supports, risking collator block-production overruns and missed slots (a liveness degradation), and per-call fees are mispriced. Inbound requires a valid BLS attestation, so there is no permissionless amplifier; the harm is a structural resource-accounting defect under honest operation. No fund loss, hence Low.

Recommended Mitigation: Add both missing dimensions to the `receive_message` benchmark: fit the signer-count term over the deployed `Linear<MinSignatures, CommitteeSize>` domain, and add a second linear com-

ponent that varies the active-set / `max_relayer_id` size the reconstruction loop iterates. Regenerate the weights and pass the active-set size into the dispatch weight closure alongside the signer count. Alternatively, cache or bound the active-set reconstruction so the per-message cost does not scale with `max_relayer_id`.

Highway: Fixed in [c66a229](#), [51d6541](#), [19a42e5](#), [db12dcb](#).

Cyfrin: Verified.

7.3.20 `receive_message` declared weight omits the dispatched payload `RuntimeCall`, breaking the weight upper-bound invariant

Description: `highway-entry::receive_message` is annotated with a weight that depends only on the signer count and returns a bare `DispatchResult` ([pallets/highway-entry/src/lib.rs#L1053](#)):

```
#[pallet::weight(T::WeightInfo::receive_message(relayer_bitmap.count_ones()))]
pub fn receive_message(/* ... */) -> DispatchResult {
```

The extrinsic then dispatches an arbitrary whitelisted `RuntimeCall` decoded from the inbound payload, via `execute_payload` ([pallets/highway-entry/src/lib.rs#L1798-L1811](#)):

```
let call = <T as Config>::RuntimeCall::decode(&mut &payload[..])?;
call.dispatch(origin).map_err(|_| Error::<T>::PayloadExecutionFailed)?;
```

The nested call's weight is never added to the declared weight, and there is no post-dispatch path to charge it because the function returns `DispatchResult` rather than `DispatchResultWithPostInfo`. The only `receive_message` benchmark is native-token-only with no payload leg, and its own comment says it avoids the payload-dispatch path ([pallets/highway-entry/src/benchmarking.rs#L253](#)).

`CheckWeight` therefore reserves only the fixed base weight before dispatch. Because post-dispatch weight can only refund below the declared amount, the inner call executes effectively free from the block weight meter's perspective, so the declared weight is not an upper bound on the actual weight consumed.

Impact: `receive_message` declares its weight as a pure function of the signer count, so the dispatched payload `RuntimeCall`'s weight is never charged. This violates the FRAME "declared weight is greater than or equal to actual weight" invariant, and it is a code-level defect independent of configuration: the committee attests only the `message_id` and never evaluates payload weight, so whatever the payload dispatches runs at the fixed base while `CheckWeight` reserves only that base.

The realized block-weight impact under the shipped configuration is small. The payload is bounded at 4 KiB ([runtime/src/configs/mod.rs#L397](#), `MaxPayloadSize = ConstU32<4096>`), so the uncharged weight is bounded, not unbounded. The deployed whitelist is `System::remark / remark_with_event` only ([scripts/constants.js#L56-L58](#)), and `remark`'s weight is roughly linear in its length, so even a full 4 KiB `remark` contributes on the order of ~0.5% of `receive_message(34)`. A block of such extrinsics therefore cannot meaningfully overrun its weight budget, and there is no realistic liveness effect as deployed.

The gap widens only under an admin configuration that is not the current one. Whitelisting a heavier call, or a wrapper such as `Utility::batch` (`pallet_utility` is wired into the runtime, [runtime/src/lib.rs#L292](#)), lets one payload carry more uncharged execution, though it is still bounded by the 4 KiB payload and, for the wrapper case, additionally gated by the relayer-origin drain issue (issue 25). So the invariant violation is a genuine bug worth fixing on its own merits, but its severity as deployed is low rather than a live block-weight DoS.

Proof of Concept: Runnable test `m32_payload_weight_omission::declared_weight_is_blind_to_payload_and_not_an_upper_bound` in `pallets/highway-entry/src/tests.rs`. It computes the charged weight with the same weights::`SubstrateWeight<Test>` code the runtime is built on, and the payload weight with the same `get_dispatch_info()` machinery a corrected `receive_message` would use, then shows the charged weight is a pure function of the signer count (blind to the payload) while the omitted payload weight is real, nonzero, and attacker-scalable, so the declared weight is not an upper bound on actual.

```
mod m32_payload_weight_omission {
    use super::*;
    use polkadot_sdk::frame_support::dispatch::GetDispatchInfo;
    use polkadot_sdk::frame_support::weights::Weight;
```

```

use polkadot_sdk::frame_system;

// The deployed runtime configures CommitteeSize = 34, so count_ones() lands in [24, 34].
const AS_SHIPPED_SIGNERS: u32 = 34;

// The weight `CheckWeight` reserves for `receive_message`, computed exactly as the
// runtime computes it. Note the signature: it takes ONLY the signer count.
fn charged_weight(signers: u32) -> Weight {
    <crate::weights::SubstrateWeight<Test> as crate::weights::WeightInfo>::receive_message(
        signers,
    )
}

// The weight of a whitelisted payload call, read from its dispatch info exactly the way
// a fixed `receive_message` would read it in order to charge for it (it currently does not).
// `System::remark` is the exact "safe-looking" whitelist example used across these
// findings; its weight is linear in the remark length, so the attacker controls it.
fn payload_weight(remark_len: usize) -> Weight {
    let call = RuntimeCall::System(frame_system::Call::remark { remark: vec![0u8; remark_len] });
    call.get_dispatch_info().call_weight
}

#[test]
fn declared_weight_is_blind_to_payload_and_not_an_upper_bound() {
    // (a) The charged weight is a pure function of the signer count: two messages with
    //      the same committee but wildly different payloads are charged identically. The
    //      block weight meter is structurally blind to the payload.
    let charged = charged_weight(AS_SHIPPED_SIGNERS);
    assert_eq!(charged, charged_weight(AS_SHIPPED_SIGNERS));

    // (b) The payload call carries real, nonzero weight that the attacker scales with the
    //      payload size.
    let w_small = payload_weight(1_024); // 1 KiB payload
    let w_large = payload_weight(256 * 1_024); // 256 KiB payload
    assert!(w_small.ref_time() > 0, "payload weight must be nonzero");
    assert!(
        w_large.ref_time() > w_small.ref_time(),
        "omitted payload weight scales with attacker-chosen payload size"
    );

    // The true execution cost is charged + payload, so `charged` is NOT an upper bound on
    // the actual weight (the FRAME invariant is declared >= actual). The excess is the
    // entire payload weight, uncharged, and grows without bound in the payload size.
    let actual_small = charged.saturating_add(w_small);
    let actual_large = charged.saturating_add(w_large);
    assert!(charged.ref_time() < actual_small.ref_time());
    assert!(actual_large.ref_time() > actual_small.ref_time());

    // The uncharged gap is exactly the payload weight the meter never sees. In the real
    // runtime `Utility::batch` (wired in) makes this per-extrinsic uncharged work
    // arbitrarily large with realistically sized inner-call vectors, so a block can breach
    // its weight budget while the meter still reads only `charged`.
    assert_eq!(actual_large.ref_time() - charged.ref_time(), w_large.ref_time());
}
}

```

Run it from the workspace root:

```
cargo test -p pallet-highway-entry m32_payload_weight_omission
```

Result: test tests::m32_payload_weight_omission::declared_weight_is_blind_to_payload_and_not_an_upper_bound ... ok (1 passed). System::remark is used because it is exactly the deployed whitelist

entry ([scripts/constants.js#L56-L58](#)) and its weight is length-linear; the mock has no Utility, but if an admin whitelisted `Utility::batch` the per-extrinsic gap would be larger (still bounded by the 4 KiB payload).

The end-to-end block-weight consequence follows by inspection of the annotation, the `execute_payload` dispatch, and the benchmark: a relayer submits `receive_message`; `CheckWeight` pre-reserves `T::WeightInfo::receive_message(count_ones)` only; `execute_payload` dispatches the decoded payload, consuming more than reserved; because the extrinsic returns `DispatchResult`, `PostDispatchInfo::actual_weight` defaults to the declared weight, so the meter records only the base; repeating across a block builds a block whose measured weight sits below its real execution cost by the omitted payload weight (~0.5% per extrinsic under the deployed remark-only whitelist, larger only if a heavier call is whitelisted).

Recommended Mitigation: Make `receive_message` return `DispatchResultWithPostInfo`, add the decoded payload call's `get_dispatch_info()` weight to the declared weight (either by decoding inside the `#[pallet::weight]` closure or by pre-charging it), and report the actual consumed weight in `PostDispatchInfo`.

This is the same pattern `pallet_utility`, `pallet_multisig`, and XCM Transact use for nested calls whose weight is not known at annotation time. The closest analog is `pallet_utility::as_derivative`, which wraps a single nested call exactly like `receive_message`'s single payload call: its `#[pallet::weight]` pre-charges the inner call's declared weight, and after dispatch it reports the real consumed weight via `PostDispatchInfo` (`pallet-utility` v40.0.0, the `as_derivative` extrinsic call_index(1), https://docs.rs/pallet-utility/40.0.0/src/pallet_utility/lib.rs.html#L253):

```
// pallet_utility::as_derivative
#[pallet::weight({
    let dispatch_info = call.get_dispatch_info();
    (
        T::WeightInfo::as_derivative()
        .saturating_add(T::DbWeight::get().reads_writes(1, 1))
        .saturating_add(dispatch_info.call_weight), // inner call weight pre-charged into the
        ↪ declared weight
        dispatch_info.class,
    )
})]
pub fn as_derivative(origin: OriginFor<T>, index: u16, call: Box<<T as Config>::RuntimeCall>)
    -> DispatchResultWithPostInfo // return type that allows post-dispatch weight
    ↪ reporting
{
    // ... set up the derivative origin ...
    let info = call.get_dispatch_info();
    let result = call.dispatch(origin);
    let mut weight = T::WeightInfo::as_derivative()
        .saturating_add(T::DbWeight::get().reads_writes(1, 1));
    weight = weight.saturating_add(extract_actual_weight(&result, &info)); // add the real inner weight
    result
        .map_err(|mut err| { err.post_info = Some(weight).into(); err }) // report actual weight on
        ↪ the error path
        .map(|_| Some(weight).into()) // and on the success path
}
```

The one adaptation for `receive_message` is that its payload arrives as raw `&[u8]`, not a decoded `Box<RuntimeCall>`, so the `#[pallet::weight]` closure must first `RuntimeCall::decode(payload)` before it can call `get_dispatch_info()` on it (or, if decoding inside the annotation is undesirable, pre-charge a conservative upper bound). The post-dispatch half, returning `DispatchResultWithPostInfo` carrying base + actual inner weight, is identical to the snippet above.

Highway: Fixed in [2ba1e58](#), [74dba1c](#), [f8d70d7](#).

Cyfrin: Verified.

7.3.21 Payload whitelist keys on raw `payload[0..2]` instead of the decoded `RuntimeCall` identity

Description: The inbound payload path parses the same bytes twice, with no cross-check:

- `extract_payload_indices` reads `payload[0]` as `pallet_index` and `payload[1]` as `call_index` ([pallets/highway-entry/src/lib.rs#L1813-L1819](#)), and ensure `payload_whitelisted` gates on those raw bytes ([pallets/highway-entry/src/lib.rs#L1821-L1834](#)).
- `execute_payload` independently does `T::RuntimeCall::decode(&mut &payload[..])` and dispatches under `RawOrigin::Signed(caller)` ([pallets/highway-entry/src/lib.rs#L1798-L1811](#)).

The pallet imports `GetCallMetadata` but never uses it, so nothing checks that the decoded call's own (`pallet_index`, `call_index`) equals the bytes the whitelist looked up. The whitelist therefore trusts an unverified assumption: that `payload[0..2]` always equals the decoded call's identity.

As-shipped verification (why this is Low, not High):

Under a standard FRAME encoding the two views coincide, and the shipped runtime is exactly that standard case, verified on-chain:

- The runtime is standard `#[frame_support::runtime]` ([runtime/src/lib.rs#L256](#)) with explicit single-byte pallet indices (`#[runtime::pallet_index(N)]`, all $N < 256$: `System=0`, `Balances=10`, `Utility=13`, `HighwayEntry=52`, ...), so `RuntimeCall` SCALE-encodes as `[pallet_index, call_index, ...args]`. No wrapper or version byte.
- The entry pallet decodes against the real runtime call (type `RuntimeCall = RuntimeCall`, [runtime/src/configs/mod.rs](#)), and the payload is a bare encoded `RuntimeCall`, not an `UncheckedExtrinsic`, so no signature / `TxExtension` / version prefix sits in front of the call bytes.
- No pallet `Call` carries a custom `Encode/Decode` or a `Call-level` `#[codec(index)]`. This was verified exhaustively across all 25 runtime pallets: the 20 upstream FRAME crates (each matched to its `Cargo.lock` version in the registry cache) and the 5 local Highway pallets all use the macro-derived `Call` codec (`#[pallet::call] + #[pallet::call_index(N)]`), with zero manual `impl Encode/Decode` for `...Call` and no divergent `#[codec(index)]` on any `Call` variant (the `#[codec(index)]` attributes that exist are on non-`Call` types, e.g. `pallet-xcm`'s `Error`, `pallet-assets`' `ExistenceReason`, and compact transaction-extension fields). No `Call` enum exceeds 256 variants (max observed `call_index` 32) and all pallet indices are < 256 , so both leading bytes are single-byte.

So today `payload[0..2]` equals the decoded (`pallet_index`, `call_index`) by construction, not by luck, and there is no live whitelist bypass. The gap is latent: it only opens if a future change breaks one of those three properties.

Impact:

- **Message stranding (liveness, the sharpest as-shipped-adjacent case).** Payload bytes are hashed into `messageId` and committee-signed, so they are frozen at emit time and decoded against whatever runtime is live at delivery time (a window bounded only by the attestation TTL, which is long). If a destination runtime upgrade removes or reindexes the (`pallet`, `call`) a pending message targets, the frozen payload no longer decodes to the intended call: the attested message either fails to decode and is permanently stranded with no recovery path, or decodes to a different call than the operator whitelisted.
- **Undocumented load-bearing invariant.** The bridge silently depends on "pallet and call indices are never reindexed or reused across upgrades." FRAME convention discourages reindexing, but nothing in the bridge enforces it, and the unusually long payload lifetime widens the exposure. Keying the whitelist on the decoded identity would remove this dependency entirely.

Not an unauthorized-execution vector. The sharpest reindex scenario, a whitelisted `System::remark` (`call_index 5`) reindexed to `System::set_code` (same `Vec arg), does not dispatch: set_code requires Root, and the payload dispatches as RawOrigin::Signed(relayer), so it reverts BadOrigin. The realistic outcome of a reindex is a stranded message, not an unapproved execution. For a reindex to cause harmful execution the reused index must name a call that is both Signed-dispatchable and harmful under the relayer's own authority, which is precisely the separate finding on a Signed Balances call draining the relayer under its own origin. That harm is attributed to that relayer-drain finding and is not double-counted here; this finding covers only the whitelist-desync mechanism and its liveness consequence.`

Proof of Concept: Runnable tests in the standalone SCALE-codec audit harness under `audit/substrate-poc/` (run `cargo test --test l37_runtime_upgrade_reindex --test l37_payload_prefix_extraction`, all 5

pass):

- [audit/substrate-poc/tests/l37_payload_prefix_extraction.rs](#): a simulated `Versioned RuntimeCall` with a wrapper byte shows the two views diverge when the encoding is non-standard. This demonstrates the mechanism, but note the shipped runtime is not this case (verified above).
- [audit/substrate-poc/tests/l37_runtime_upgrade_reindex.rs](#): `call_removed_strands_the_message` is the real, in-scope impact (a removed/reindexed target strands the frozen, attested payload). The companion `frozen_payload_reindexes_to_dangerous_call...` test should be read as proving decode-divergence and whitelist-pass only; to demonstrate actual harmful execution it must target a Signed-dispatchable call (drop the `set_code` example, which fails `BadOrigin`), at which point the harm is that of the separate relay-drain finding.

Recommendation:

Validate the decoded call's identity, not the raw prefix. After `RuntimeCall::decode`, use the already-imported `GetCallMetadata` to read the decoded call's `pallet_index / call_index` and check those against the whitelist (and/or re-encode the decoded call and compare its prefix to the input). This is a cold-path, per-payload-once cost and removes the never-reindex invariant dependency: a message that decodes to a non-whitelisted call is rejected regardless of how the encoding evolves. Separately, document the payload-lifetime / reindex invariant and consider rejecting payloads that do not round-trip, so a reindex produces a clean rejection rather than a silent decode into a different call.

Highway: Fixed in [75527d7](#).

Cyfrin: Verified.

7.3.22 `RelayerRegistry::update_bls_key` overwrites a seated relay's BLS key with no pin-window coordination, invalidating in-flight attested proofs

Description: `verify_bls_attestation` reconstructs the committee's aggregate public key **live** at claim time: for each set bit in the pinned committee bitmap it reads `RelayerRegistry::relayer_bls_key(id)` ([highway-entry:1341](#)), parses and aggregates the pubkey ([highway-entry:1345](#)), and checks the aggregate signature against that reconstructed set ([highway-entry:1348](#)). The verifier honors a pin window: a proof may claim against the current epoch or the retained `current - 1` epoch ([highway-entry:1277-1280](#)). The committee aggregate is produced off-chain from the keys current at signing time; the on-chain check reads each key live at claim time.

`update_bls_key` changes that live key state out from under an in-flight proof. Gated only by `ensure_admin_or_registrar` ([highway-registry:1184](#)), it overwrites `RelayerBlsKeys[id]` with a new key for **any** existing relay ([highway-registry:1213](#)), including one currently seated, with no active-set or pin-window guard at all. On delivery the reconstruction loop reads the **new** pubkey, so the reconstructed aggregate no longer matches the off-chain signature aggregated over the **old** key, and the aggregate check at [highway-entry:1348](#) fails. Because there is no guard, this invalidates proofs pinned to the current epoch as well as `current - 1`.

The sibling mutator `remove_relayer` strands proofs by the same root cause but by *deleting* the key rather than overwriting it; that case is issue 5 and is not re-filed here. Shared root cause: the registry mutates a relay's BLS-key state with guards keyed to the active-set head, while `verify_bls_attestation` reads that key live against a pin window that extends past the head to `current - 1`.

Impact: Bounded liveness, not fund loss. A rotation on a still-referenced seated relay invalidates every in-flight, already-attested inbound message whose pinned committee includes that relay: on delivery the reconstructed aggregate uses the new key, the aggregate check fails, and the message reverts before the `ExecutedInboundMessages` marker commits, so it is re-deliverable and the committee re-attests under the next epoch. No funds are lost and no invalid signature is accepted. The affected window is `current` and `current - 1` (strictly broader than the removal case in issue 5, which is `current - 1` only).

The Highway team's position is that immediate key invalidation is intended: `update_bls_key` exists to rotate a **compromised** key, so it must take effect at once, and gating it behind the active-set check would keep a leaked key valid until the relay cycles out of the set, which is strictly worse. That is correct for emergency rotation. The finding stays at Low rather than closed because there is no path for **lossless routine key rotation**: rotating a healthy key always forces a one-epoch re-attestation of in-flight traffic, since emergency and routine rotation share

one mechanism with the same old-key handling. It is also a design inconsistency specific to this runtime, which added committee pinning (retained per-epoch bitmaps + the `current / current - 1` window) precisely so an in-flight proof survives an epoch rotation *without* re-attestation, yet the live key read means any key overwrite still forces re-attestation, so the pin window under-delivers for key changes while advertising `current - 1` claimability.

Proof of Concept: By inspection of the mutator and the verifier:

1. `update_bls_key` ([highway-registry:1178-1216](#)) validates the new key and PoP, then writes `RelayerBlsKeys::<T>::insert(id, &new_bls_key)` ([highway-registry:1213](#)) with no active-set check.
2. `verify_bls_attestation` reads `relayer_bls_key(id)` live for each committee seat ([highway-entry:1341](#)), aggregates it ([highway-entry:1345](#)), and verifies the aggregate ([highway-entry:1348](#)), honoring both current and the retained `current - 1` epoch ([highway-entry:1277-1280](#)).
3. Sequence: relayer R is seated in the current committee; a message is attested (aggregate signed with R's current key). Before delivery an admin/registrar calls `update_bls_key(R, new_key)`. On delivery the reconstruction reads R's new key, the aggregate no longer matches the old-key signature, and the message reverts at the aggregate check. It is re-deliverable once the committee re-attests under the next epoch.

A runnable mock-runtime test mirrors the committee-key path used by the inbound tests: seat a committee, build a valid aggregate over the current keys, `update_bls_key` on a seated relayer, deliver, and assert the aggregate verify fails.

Recommended Mitigation: Acknowledged as intended on the EVM leg. The identical `update_bls_key` issue was ruled won't-change there, "must be able to rotate a leaked BLS key; gating is strictly worse" ([evm-06](#)); expect the same disposition here. The actionable item is to **document** that `update_bls_key` invalidates in-flight proofs for the affected committee, which re-attest the next epoch. If lossless routine rotation is wanted, the machinery exists: pin keys to the signing epoch and revoke emergencies via the existing `MinValidEpoch` floor (making emergency revocation epoch-wide rather than per-relayer). Do **not** gate `update_bls_key` behind the active set, which delays compromised-key revocation.

Highway: Acknowledged, [PR 89](#) is documentation only, which is what this finding asked for.

Cyfrin: Rationale accepted with a condition: immediate rotation of a compromised BLS key is a reasonable fail-closed tradeoff, but the "no funds lost" conclusion depends on Highway reliably identifying and re-attesting every affected in-flight message under an accepted epoch and valid TTL; the Substrate pallet preserves retryability but does not perform that recovery.

7.3.23 `complete_undelegation` charges the flat worst-case weight and shifts with $O(n)$ remove

Description: `complete_undelegation` removes an NFT from a relayer's delegation vector with a linear scan and shift ([pallets/highway-nft-delegation-registry/src/lib.rs:493-497](#)):

```
RelayerDelegations::<T>::mutate(delegation.relayer_id, |delegations| {
    if let Some(pos) = delegations.iter().position(|&id| id == nft_item) {
        delegations.remove(pos); // Vec::remove is O(n) shift
    }
});
```

Both `.position(...)` and `Vec::remove(pos)` are linear in `delegations.len()`, bounded by `T::MaxDelegationsPerRelayer`, which the delivered runtime sets to 1000 ([runtime/src/configs/mod.rs:534](#)).

The declared weight is the flat worst case with no refund ([lib.rs:475](#)), so every call is charged for a 1000-element scan-and-shift regardless of the actual list length. The sibling `delegate` does refund: it declares the worst case ([lib.rs:329](#)), snapshots the real length before mutating ([lib.rs:393-394](#)), and returns the length-proportional weight ([lib.rs:433](#)). The refund pattern is available in the same pallet and simply not applied here.

Impact: Low severity: a weight-accounting and efficiency defect, not a fund-loss or authorization issue, and bounded by the `MaxDelegationsPerRelayer <= 1000` cap. Callers overpay for short lists, and if the benchmark for the 1000-element case is itself inaccurate the flat charge enables block-fill griefing. A new `integrity_test` ([lib.rs:103-113](#)) pins `MaxDelegationsPerRelayer <= 1000`, forcing a benchmark re-run before the constant is raised, but it does nothing for the $O(n)$ shift or the missing refund.

Recommended Mitigation: Replace `remove(pos)` with `swap_remove(pos)` if delegation order is irrelevant (only membership is consulted elsewhere), bringing the removal step to $O(1)$ ([lib.rs:495](#)):

```
RelayerDelegations::<T>::mutate(delegation.relayer_id, |delegations| {
    if let Some(pos) = delegations.iter().position(|&id| id == nft_item) {
-       delegations.remove(pos);
+       delegations.swap_remove(pos);
    }
});
```

Optionally mirror delegate's length-proportional refund so short lists are not charged the flat worst case. This matters because `swap_remove` only removes the $O(n)$ shift; the `.position(...)` lookup is still a linear scan, so the extrinsic stays $O(n)$ and the flat charge still over-bills a short list for a full-length scan. `delegate` already implements the refund in this pallet, and it is three small changes:

1. Keep the worst-case `#[pallet::weight]` declaration so `CheckWeight` still pre-reserves the maximum ([lib.rs:329](#)).
2. Return `DispatchResultWithPostInfo` instead of the current bare `DispatchResult` ([delegate at lib.rs:334](#) vs [complete_undelegation at lib.rs:476](#)).
3. Snapshot the real vector length before the mutation ([lib.rs:394](#)) and return that length-proportional weight in the `PostDispatchInfo`, so the meter refunds `worst_case - actual` ([lib.rs:433](#)).

Applied to `complete_undelegation` (composed with the `swap_remove` change above):

```
-     pub fn complete_undelegation(origin: OriginFor<T>, nft_item: T::ItemId) -> DispatchResult {
+     pub fn complete_undelegation(origin: OriginFor<T>, nft_item: T::ItemId) ->
↪     DispatchResultWithPostInfo {
        // ... existing checks, `delegation` resolved ...
+        // Snapshot length before removal for the refund (mirrors `delegate`).
+        let current_len = RelayerDelegations::<T>::get(delegation.relayer_id).len() as u32;
        RelayerDelegations::<T>::mutate(delegation.relayer_id, |delegations| {
            if let Some(pos) = delegations.iter().position(|&id| id == nft_item) {
                delegations.swap_remove(pos);
            }
        });
        // ... existing event emission ...
-        Ok(())
+        Ok(Some(T::WeightInfo::complete_undelegation(current_len)).into())
    }
```

`WeightInfo::complete_undelegation` already takes the length parameter (the declaration passes `MaxDelegationsPerRelayer`), so no new benchmark is needed; the declared weight is unchanged and only the reported post-dispatch weight shrinks to the actual list length.

Highway: Fixed in [a2418c1](#).

Cyfrin: Verified.

7.3.24 Proof-of-Possession reuses the message-signing ciphersuite/DST instead of a dedicated `_POP_-DST`

Description: The aggregate BLS scheme relies on a Proof-of-Possession (PoP): at registration a relayer signs its own public key to prove it holds the matching private key, which is what defends the aggregate against rogue-key attacks. The IRTF CFRG BLS signature draft ([draft-irtf-cfrg-bls-signature](#)) binds PoP verification to a *distinct* ciphersuite/DST from message signing, so a signature in one role is cryptographically non-transferable to the other:

- message-signing DST (the codebase's Basic ciphersuite, trailing `_NUL_` scheme tag): `BLS_SIG_-BLS12381G2_XMD:SHA-256_SSWU_RO_NUL_`

- IRTF PoP DST (the BLS_POP_ prefix plus the trailing _POP_ scheme tag): BLS_POP_BLS12381G2_XMD:SHA-256_SSWU_RO_POP_

The registry uses the *signing* DST for both. `verify_pop` passes the one BLS_DST constant `b"BLS_SIG_-BLS12381G2_XMD:SHA-256_SSWU_RO_NUL_"` ([pallets/highway-registry/src/lib.rs#L120](#)) into its `sig.verify(...)` ([pallets/highway-registry/src/lib.rs#L702](#)); there is no dedicated `_POP_` ciphersuite. The domain separator meant to keep a PoP signature and a message signature from ever being confused is therefore absent.

Impact: Safe under the current code, but by coincidence rather than by design. The separation instead comes from the two message spaces being disjoint: a PoP preimage is a 32-byte keccak digest of `domain || pubkey`, while the actual signing payload is `messageId(32) ++ ttl_LE(4) ++ slot_LE(4) ++ relayerId_LE(4) = 44` bytes. A 32-byte input can never equal a 44-byte one, so hash-to-curve maps them to different points despite the shared DST, and a PoP signature cannot be replayed as a message attestation, nor the reverse.

The fragility is that this safety rests on the two formats happening to differ, not on the cryptographic separator that should enforce it. Any future change that extends the PoP preimage or shrinks the message layout could make the spaces overlap, at which point a signature gathered in one role would verify in the other (signature confusion). A dedicated `_POP_` DST removes the dependency permanently: the signatures become non-transferable between roles by the domain separator itself, independent of the preimage shape.

Proof of Concept: N/A: not exploitable under the current code, since the PoP and message preimages are disjoint by length.

Recommended Mitigation: Add a dedicated PoP DST constant and use it only in the PoP verification path:

```
// IRTF draft-irtf-cfrg-bls-signature 4.2.3: the PoP DST carries the `BLS_POP_`
// prefix and the trailing `_POP_` scheme tag (not the Basic `_NUL_` tag).
const BLS_POP_DST: &[u8] = b"BLS_POP_BLS12381G2_XMD:SHA-256_SSWU_RO_POP_";
```

Pass `BLS_POP_DST` (instead of `BLS_DST`) at the `verify_pop sig.verify(...)` site ([lib.rs:702](#)), leaving message verification on `BLS_DST`. This is the registered PoP ciphersuite, separated from the `_NUL_` signing DST in both the prefix and the suffix. The cost is one extra constant and one path.

Highway: Fixed in [617b275](#), [4c69daa](#).

Cyfrin: Verified.

7.3.25 `clear_relayer_delegations` fail-fasts on one un-unbindable NFT, bricking admin recovery

Description: `clear_relayer_delegations` is the documented admin recovery path for a relayer's delegations. It iterates every delegation and calls `unbind` per item with ? ([pallets/highway-nft-delegation-registry/src/lib.rs:661-674](#)):

```
for nft_item in nft_items.iter() {
    if let Some(info) = Delegations::<T>::take(nft_item) {
        T::NftBinder::unbind(&info.delegator, nft_item)?; // fail-fast
        PendingUndelegations::<T>::remove(nft_item);
    }
}
```

A single failing `unbind` aborts the whole call. Combined with the separate issue where a foreign-owned lock makes `unbind` fail (its `enable_transfer` fails), one un-unbindable item bricks the recovery path for the entire relayer, leaving it half-cleared. The doc comment promises an admin recovery guarantee the implementation does not deliver.

Impact: Admin teardown DoS. If any single NFT in the list cannot be unbound (foreign lock, runtime hook rejection), the entire `clear` aborts and the relayer is stuck in a half-cleared state, with no way for the admin to complete cleanup of the items that can be cleared.

Proof of Concept: Not included, structural. The fail-fast is direct from the ? on `unbind` ([lib.rs:670](#)).

Recommended Mitigation: Make the loop resilient to per-item `unbind` failures: collect failures instead of aborting, and read each record with `get` rather than `take` so it is only removed once its `unbind` succeeds. That keeps the

drift-avoidance the original fail-fast comment (lib.rs:666-669) was protecting (this pallet's Delegations map and pallet-highway-nft-permission's NFT lock stay in sync) while still completing cleanup for every item that can be cleared (lib.rs:661-674):

```
+ // Best-effort teardown: collect items we cannot unbind instead of aborting.
+ let mut failed: Vec<T::ItemId> = Vec::new();
+ for nft_item in nft_items.iter() {
-     if let Some(info) = Delegations::<T>::take(nft_item) {
-         T::NftBinder::unbind(&info.delegator, nft_item)?;
-         PendingUndelegations::<T>::remove(nft_item);
+         // `get`, not `take`: do not consume the record until the unbind succeeds,
+         // so a failed item stays consistent (record present == NFT still bound) and retryable.
+         if let Some(info) = Delegations::<T>::get(nft_item) {
+             match T::NftBinder::unbind(&info.delegator, nft_item) {
+                 Ok(()) => {
+                     Delegations::<T>::remove(nft_item);
+                     PendingUndelegations::<T>::remove(nft_item);
+                 }
+                 // Leave the record in place and report it rather than aborting the call.
+                 Err(_) => failed.push(*nft_item),
+             }
+         }
+     }
+ }
+ if !failed.is_empty() {
+     Self::deposit_event(Event::ClearPartiallyFailed { relayer_id, failed });
+ }
```

This delivers the best-effort admin teardown the documentation describes without reintroducing the record/lock drift the fail-fast was avoiding.

Highway: Acknowledged. Treated as a privileged-admin state-consistency scenario, not externally reachable: ordinary users and relayers cannot touch the Pallet-namespaced TransferDisabled attribute, and the fail-fast would require the Highway admin (ForceOrigin = EnsureHighwayAdmin) to mutate or remove it while the delegation record remains bound, violating the documented collection invariant. This is enforced operationally (admins must not alter protocol-owned collection state outside the permission pallet), so the best-effort cleanup is defense-in-depth rather than a required fix; the implementation PR is closed as not required for the deployed threat model.

Cyfrin: Rationale accepted with a condition: the fail-fast recovery is acceptable only while the Highway NFT permission pallet remains the sole writer of the Pallet-namespaced TransferDisabled state and the trusted Highway admin never changes or removes bound collection state out of band; if that wiring or trust policy changes, implement retryable per-item cleanup.

7.3.26 A public Proof-of-Possession can be replayed to re-seat a freed BLS key onto a different relayer

Description: The Proof-of-Possession preimage is bound only to the key, with no nonce, relayer id, chain id, or expiry (pallets/highway-registry/src/lib.rs#L677-L709):

```
fn compute_pop_message(compressed_key: &BlsPublicKeyCompressed) -> [u8; 32] {
    // keccak256(POP_DOMAIN || compressed_key_bytes)
}
fn verify_pop(public_key, signature, compressed_key) -> Result<(), Error<T>> {
    let pop_message = Self::compute_pop_message(compressed_key);
    // verify signature over pop_message
}
```

PoP signatures are submitted as extrinsic arguments to register_relayer and update_bls_key, so they are permanently public in block data. Because the same key always yields the same preimage, an old PoP is indefinitely reusable.

remove_relayer frees the key-to-relayer mappings (pallets/highway-registry/src/lib.rs#L963-L964), and both admission paths re-check only BlsKeyToRelayer::contains_key for the BlsKeyAlreadyInUse guard

([pallets/highway-registry/src/lib.rs#L835-L836](#) and [pallets/highway-registry/src/lib.rs#L1193-L1213](#)), which passes once the key is free.

Impact: Because the PoP proves only knowledge of the key, not intent to bind it to a particular relayer at a particular time, a PoP published during one registration stays usable after the key is freed. Once `remove_relayer` clears `BlsKeyToRelayer[K]`, both admission paths accept the original public P again: `verify_pop(K, P, K)` still holds (the preimage is just `keccak256(POP_DOMAIN || K)`) and `BlsKeyAlreadyInUse` no longer fires. So the admin or any member of the lower-trust `AuthorizedRegistrars` set can re-seat K onto a different relayer id with no fresh possession proof, undercutting the "removal requires explicit re-authorization" guarantee the pallet documents. If the original holder kept K's private key, a removed or banned signing key can re-enter committees under a new id.

This is a distinct consequence from the other PoP-preimage findings (freshness / domain binding, parsed-key vs compressed-bytes agreement) and is not covered by them. Severity is Low: it needs the admin or a registrar role, but a *compromised registrar*, not only the admin, suffices, which is what weakens the anti-rogue-key defense.

Proof of Concept: By inspection of the preimage, the removal path, and the admission checks:

1. `register_relayer(A, K, P)` succeeds; P is recorded in block data.
2. `remove_relayer(A)` frees K (`BlsKeyToRelayer::remove(K)`, `RelayerBlsKeys::remove(A)`).
3. A registrar submits `update_bls_key(B, K, P)` (or `register_relayer` for a new id with (K, P)), reusing the public P.
4. `verify_pop(K, P, K)` returns Ok (preimage is `keccak256(POP_DOMAIN || K)`), `BlsKeyAlreadyInUse` does not fire, and K is bound to B.

A runtime test would register with (K, P), remove the relayer, then assert that re-registration or `update_bls_key` with the identical (K, P) succeeds, demonstrating the PoP is replayable with no fresh proof.

Recommended Mitigation: Bind the PoP preimage to fresh, non-reusable context: include the target `relayer_id` and a per-relayer monotonic nonce (or the current block/epoch) in `compute_pop_message`, and require a new PoP for every key assignment (registration and `update_bls_key`). This also composes with the separately recommended PoP mitigations (a dedicated PoP domain and consistent cross-chain encoding).

For reference, the EVM leg builds the same key-only preimage `keccak256(BLS_POP_DOMAIN || blsPublicKey)` in [RelayerRegistryLogic._verifyPop](#), so the same freshness and id binding should be mirrored there to keep the two legs consistent.

Highway: Acknowledged, treated as documentation-only. Reusing a published PoP still requires admin or authorized-registrar admission; the re-registered relayer starts unauthorized and is not automatically placed in an active set; and PoP freshness would not remove capability from a holder of the private key, who can always sign a fresh PoP, so replay does not weaken the anti-rogue-key property. The admission controls and the fact that removal frees the key mapping will be documented.

Cyfrin: Rationale accepted with a condition: the admin and authorized-registrar roles must be trusted to assign and rotate BLS keys even for already active relayers; replay grants no private-key capability, but `update_bls_key` can bind a freed key to an already authorized active relayer without the separate reauthorization and active-set steps that apply to fresh registration.

7.3.27 **Permissionless** `pallet_assets` creation plus no asset-control binding in `register_token` lets a third party own a bridged asset

Description: The runtime wires `pallet_assets` with a permissionless `CreateOrigin` ([runtime/src/configs/mod.rs#L192-L194](#)):

```
type CreateOrigin =  
    frame_support::traits::AsEnsureOriginWithArg<frame_system::EnsureSigned<AccountId>>;
```

so any signed account can `Assets::create(id, ...)` for any u32 asset id (paying a refundable deposit) and become that asset's owner, admin, issuer, and freezer.

`highway-config::register_token` stores an `asset_id` for a bridged token but never verifies that the asset exists or that the bridge/admin controls it ([pallets/highway-config/src/lib.rs#L525-L564](#)). The inbound Mint path later calls the low-level fungibles `mint_into(asset_id, recipient, amount)` on that id, which mints regardless of who owns the asset team.

Impact: Asset ids are attacker-choosable `u32` values and creation is permissionless, so an attacker can front-run the admin: create the asset id the admin intends to wrap before `register_token` runs, taking the owner/admin/issuer/freezer roles. The corridor is then configured against an attacker-owned asset. `mint_into` still mints for legitimate inbound transfers, but the attacker can then `freeze_asset`, `freeze(id, user)`, or `start_destroy + destroy_accounts` on the bridged asset, freezing or wiping users' bridged balances. A corridor pointed at a non-existent `asset_id` additionally causes a per-message inbound DoS (MintFailed) and outbound Burn/Escrow failures.

Severity is Low because a careful admin who creates the asset first (or does create + register atomically via `utility.batch`) wins the race and holds the team keys; the freeze/destroy damage requires losing that race.

Proof of Concept: By inspection of the runtime `CreateOrigin` and `register_token`:

1. The admin announces or an observer anticipates that asset id A will back a bridged token.
2. Attacker calls `Assets::create(A, attacker, min_balance)` and becomes owner/admin/issuer/freezer of A.
3. Admin calls `register_token(token_id, ..., asset_id = A, ...)`, which succeeds with no asset-existence or ownership check, and configures the corridor.
4. Users bridge in; `mint_into(A, user, amount)` mints their wrapped balance.
5. Attacker calls `Assets::freeze_asset(A)` or `Assets::start_destroy(A) + destroy_accounts(A)`, freezing or wiping every user's bridged balance of A.

Recommended Mitigation: For a bridge that mints wrapped assets, gate `pallet_assets::CreateOrigin` to root or the Highway admin rather than `EnsureSigned`. In addition, have `register_token` (and `configure_token_bridge`) assert that the asset exists and that its admin/issuer/freezer are the bridge pallet account or the Highway admin, so a corridor can never be configured against an asset the protocol does not control.

Highway: Fixed in [877226a](#).

Cyfrin: Verified.

7.3.28 `pallet_nfts` `bound` type `WeightInfo = ()` with permissionless `CreateOrigin` enables zero-weight block stuffing by any account

Description: The runtime binds the `pallet_nfts` `WeightInfo` associated type to the unit type, `type WeightInfo = ()`; (commented "Use default weights for now", [runtime/src/configs/mod.rs#L504](#)). The `()` impl returns `Weight::zero()` for every `pallet_nfts` extrinsic, so each mint, transfer, burn, `set_attribute`, `set_metadata`, `approve_transfer`, `cancel_approval`, and the rest is declared as costing zero `ref_time` and zero `proof_size`, regardless of the real storage work it performs.

The same `impl pallet_nfts::Config for Runtime` sets `type CreateOrigin = AsEnsureOriginWithArg<frame_system::EnsureSigned<AccountId>>`, so collection creation is permissionless: any signed account can create a collection (reserving `CollectionDeposit`) and mint items into it as that collection's own issuer, with no bridge permission NFT and no privileged role required ([runtime/src/configs/mod.rs#L479-L507](#), and `CreateOrigin` at [runtime/src/configs/mod.rs#L485](#)).

Weight is the mechanism that bounds per-block work and prices transactions. With these calls reporting zero weight, the block builder treats them as free and packs them up to the block length limit (the only remaining cap), while their true execution and PoV cost goes unaccounted. An attacker:

1. calls `create` once (permissionless, `EnsureSigned`), reserving `CollectionDeposit`;
2. mints one or two items into the new collection (they are the issuer);

3. then loops zero-weight, no-additional-deposit storage mutations indefinitely, for example `approve_transfer` then `cancel_approval` on the same item, transfer between two attacker accounts, or `set_attribute` then `clear_attribute`.

Each looped call mutates storage (real `ref_time` and PoV) but charges only the base and length fee at zero weight. Deposits bound only the state-creating operations; the loopable approval and transfer operations take no per-call deposit, so the spam is not deposit-limited.

Impact: A permissionless liveness / denial-of-service surface. Because each spam call carries zero weight, a single funded account can fill a block with operations whose aggregate real execution and proof size far exceed the weight budget the chain sized its slot against. Two failure modes:

- Block execution and import overrun the slot, degrading liveness as collators and validators fall behind.
- On this Cumulus parachain, zero `proof_size` means the PoV budget is never decremented, so a packed block's actual PoV can exceed the relay chain's `max_pov_size` and be rejected as invalid, so the parachain fails to produce a valid block for that slot.

No funds are lost or minted; the impact is availability. Throttling is limited to the base plus length fee and the block length limit, none of which prices the compute or PoV the calls actually consume, which is precisely the job weight is meant to do. The reachability is the load-bearing point: `CreateOrigin = EnsureSigned` makes this permissionless rather than gated on holding a bridge permission NFT, so it is not limited to relayers or governance participants.

Proof of Concept: Observable by inspection of the `impl pallet_nfts::Config for Runtime` block: `CreateOrigin = EnsureSigned` together with `WeightInfo = ()` ([runtime/src/configs/mod.rs#L479-L507](#), `CreateOrigin` at [runtime/src/configs/mod.rs#L485](#), `WeightInfo` at [runtime/src/configs/mod.rs#L504](#)). A runtime test that submits a batch of `approve_transfer` / `cancel_approval` calls on a self-minted item and inspects the dispatched weight (zero) demonstrates the under-charge.

Recommended Mitigation: Primary fix: bind the upstream-generated weights (`pallet_nfts` ships them) instead of `()`, so every NFT extrinsic is metered for its real `ref_time` and PoV, which removes the denial-of-service surface. The one-line change at [runtime/src/configs/mod.rs#L504](#):

```
- type WeightInfo = (); // Use default weights for now
+ type WeightInfo = pallet_nfts::weights::SubstrateWeight<Runtime>;
```

Note `pallet_highway_nft_permission` also still binds `type WeightInfo = ()` ([runtime/src/configs/mod.rs#L529](#)), but its only extrinsic (`mint_permission_token`) is `EnsureHighwayAdmin`-gated, so the zero-weight spam is not permissionlessly reachable there; that is an admin-only hygiene issue worth fixing for correctness, not this finding's DoS.

The primary fix removes the DoS regardless of `CreateOrigin`: once weights are metered, permissionless collection creation is a normal, safe configuration. Separately, confirm the intended `CreateOrigin` policy with the team: `EnsureSigned` (permissionless creation) is fine with real weights; restrict it to the highway admin only if collection creation is meant to be private to the bridge permission collection.

Because `WeightInfo` and `CreateOrigin` are runtime-level bindings (set where the pallets are assembled into a runtime, not in the pallet itself), apply and verify this fix in the deployed production runtime, not only in this audited runtime: the production runtime's bindings are authoritative and may differ from what is bound here.

Highway: Fixed in [a756dac](#).

Cyfrin: Verified.

7.3.29 `remove_chain` becomes permanently un-callable once `TokenChainConfig` exceeds `RemoveChainMaxScan`

Description: `highway-config::remove_chain` scans the entire `TokenChainConfig` double-map and bails with `TooManyTokenChainConfigs` at the scan cap ([pallets/highway-config/src/lib.rs#L1072-L1095](#)):

```
let cap = T::RemoveChainMaxScan::get();
for (_, cid, _) in TokenChainConfig::<T>::iter() {
```

```

    if scanned >= cap {
        return Err(Error::::TooManyTokenChainConfigs.into());
    }
    scanned = scanned.saturating_add(1);
    if cid == chain_id { has_token_configs = true; break; }
}
ensure!(!has_token_configs, Error::::ChainHasTokenConfigs);

```

The loop breaks early only when it finds an entry whose chain matches the target. But a chain can only be removed when it has no token configs (otherwise the subsequent `ensure!` returns `ChainHasTokenConfigs`). So the only path that actually removes a chain is the one where the target chain has no entries, which means the loop never breaks early and scans the whole map.

`RemoveChainMaxScan` is 1000 ([runtime/src/configs/mod.rs#L376](#)).

Impact: Once `TokenChainConfig` holds more than 1000 total (token, chain) pairs, any config-less chain becomes impossible to deregister: the scan reaches the cap and reverts `TooManyTokenChainConfigs` before it can confirm the chain has no configs. That map size is reachable in normal operation (for example 40 tokens across 26 chains is 1040 entries). The chain then cannot be removed until the admin prunes the global config count back below 1000.

This is admin-only, causes no fund loss, and is recoverable by pruning, hence Low. Unlike the separately reported `integrity_test` panic from a benchmark-bound mismatch, this is not such a panic: the benchmark is `Linear<0, 1000>` and matches the cap, so the runtime's own integrity assertions pass.

Proof of Concept: By inspection of the loop and the cap:

1. Register chain X with no token corridors.
2. Populate `TokenChainConfig` with more than 1000 total entries across other chains (normal multi-token, multi-chain operation).
3. Call `remove_chain(X)`. The loop iterates other chains' entries, never matches X, reaches `scanned == 1000`, and returns `TooManyTokenChainConfigs`.
4. X can never be removed until the admin deletes enough token configs elsewhere to bring the map under 1000.

A runtime test would insert 1001 `TokenChainConfig` entries for chains other than X, then assert `remove_chain(X)` reverts `TooManyTokenChainConfigs` despite X having no configs of its own.

Recommended Mitigation: Iterate only the target chain's entries rather than the whole map: maintain a per-chain config count (incremented/decremented in `configure_token_bridge` / the corridor removal path) or a (chain_id, token_id)-keyed index, and have `remove_chain` check that count. Then the cost is bounded by the chain's own configs and a large global map cannot block removal of an unrelated, config-less chain.

Highway: Acknowledged. `remove_chain` reverts only when the total `TokenChainConfig` count exceeds `RemoveChainMaxScan` (1000); below that the scan completes and a config-less chain is always removable. The deployed configuration is a curated set of a few tokens across a few chains (well under 20 pairs) with no roadmap approaching a fraction of 1000, so the scan never reaches the cap.

Cyfrin: Rationale accepted with a condition: the global `TokenChainConfig` population must remain at or below the benchmarked `RemoveChainMaxScan` limit of 1000; before permitting growth beyond that bound, Highway must raise and re-benchmark the limit or replace the global scan with a per-chain count or index.

7.3.30 The highway-nft-permission soft-lock model has four related lock-integrity and stale-ownership gaps across bind/unbind and the delegation registry

Description: `highway-nft-permission::bind` does not take custody of a delegated NFT; it places a soft lock by calling `pallet_nfts Transfer::disable_transfer` (which sets the `TransferDisabled` system attribute) and records `BoundTokens[item] = account_id` (bind at [pallets/highway-nft-permission/src/lib.rs#L329-L360](#), `disable_transfer` at [pallets/highway-nft-permission/src/lib.rs#L348](#)). `unbind` clears that one attribute via `enable_transfer` ([pallets/highway-nft-permission/src/lib.rs#L386](#)). The delegation registry then caches the

owner-at-delegation-time as `delegation.delegator` and uses that cached value as the sole authority for the entire undelegation lifecycle (delegate at [pallets/highway-nft-delegation-registry/src/lib.rs#L397](#), cached delegator at [pallets/highway-nft-delegation-registry/src/lib.rs#L408](#)). Four related defects follow from this soft-lock-plus-cached-owner design. Each is a real code fact, and each is latent in the delivered runtime for one shared reason: the soft lock, `pallet_nfts::Config::Locker = ()`, and the absence of any force-transfer path together make owner drift while an item is bound unreachable, and no second component shares the collection through the nonfungibles `Transfer` interface.

1. `unbind` re-enables transfer without checking current ownership, asymmetric with `bind`. `bind` reads the current `pallet_nfts` owner and enforces `WrongOwner` ([pallets/highway-nft-permission/src/lib.rs#L336-L340](#)); 'unbind' checks only `'BoundTokens[item] == account_id'` ([pallets/highway-nft-permission/src/lib.rs#L380](#)), that is, the account that originally bound the item, never that it still owns it ([pallets/highway-nft-permission/src/lib.rs#L374-L391](#)).
2. The delegation registry keys authorization to the delegate-time owner rather than the current owner. `undelegate` requires `delegation.delegator == who` ([pallets/highway-nft-delegation-registry/src/lib.rs#L450](#)), `cancel_undelegation` the same ([pallets/highway-nft-delegation-registry/src/lib.rs#L539](#)), `complete_undelegation` passes `delegation.delegator` to `unbind` ([pallets/highway-nft-delegation-registry/src/lib.rs#L491](#)), and the `SlashOrigin` teardown `clear_relayer_delegations` likewise ([pallets/highway-nft-delegation-registry/src/lib.rs#L670](#)). Because `delegate` soft-locks rather than escrows, the cached delegator diverges from the current owner if any owner-change path exists that the soft lock does not cover.
3. `bind` swallows `ItemLocked`. If the item is already locked, `bind` treats the failed `disable_transfer` as success and still writes `BoundTokens[item] = account_id` (the tolerated match arm inside `bind`, [pallets/highway-nft-permission/src/lib.rs#L329-L360](#)). `BoundTokens` can then record a binding the pallet never actually locked, and a later `unbind` calls `enable_transfer` unconditionally, clearing a lock the pallet never placed.
4. `bind` / `unbind` manage only one of `pallet_nfts`' four independent transfer-lock conditions (the `Transfer-Disabled` system attribute). The `Freezer` role's item-level `Transferable` lock (set by `lock_item_transfer`) is a different flag that `bind` never consults and `unbind` cannot clear, so `bind` is blind to a `Freezer` lock and `unbind` cannot undo one.

Impact: These are lock-integrity and stale-ownership correctness gaps on the NFT collateral lifecycle. In the delivered runtime each is latent: while an item is bound its owner cannot change (the soft lock blocks transfer, `Locker = ()`, and `pallet_nfts` exposes no item force-transfer), so `BoundTokens[item]` and the current owner cannot diverge, and no co-resident pallet shares the collection through the `Transfer` interface. The cluster is therefore defense-in-depth: the safety of the entire binding/undelegation flow rests on the current lock model holding, and every item below reactivates under any lock-model change (escrow removed, a `Locker` hook added, a force-transfer path introduced, or a second pallet wired onto the same collection).

The consequences that would then bite:

- A stale-delegator account drives the whole undelegation lifecycle: `complete_undelegation` calls `unbind(delegator, item)` and succeeds because `BoundTokens[item] == delegator` regardless of who holds the NFT now; for a Path-A relayer-tier NFT this same step also fires `deauthorize_operator(relayer_id)` off the stale account's action, and the new owner has no on-chain remedy.
- `unbind` re-enables transfer on an item the original binder no longer owns; a buyer who acquired the NFT assuming the lock held the delegation inherits a now-transferable item in a state the protocol did not expect.
- `bind` adopting a foreign lock is a confused-deputy violation: a later `unbind` clears a lock the pallet never owned, or the registry believes the item is bound while the lock is actually foreign, so `BoundTokens` misrepresents the real lock state.
- The `Freezer` case is an operational hazard: an admin freeze meant to take an item out of circulation is silently ignored and `bind` delegates it anyway; after `unbind`, Highway's view (freed and transferable) diverges from on-chain reality (still `Freezer`-locked), and only the `Freezer` can clear it, so the holder has no Highway-facing recovery.

Severity Low: no funds move, all four are latent under the as-shipped lock model, and the `Freezer` case additionally requires a privileged, already-trusted role. The grouping is filed as one finding because the four share a single root

(soft-lock plus cached owner) and one mitigation direction.

Proof of Concept: The drift-unreachability that makes defects 1 and 2 latent is shown by the client's own `bind_transfer_is_locked` ([pallets/highway-nft-permission/src/tests.rs](#)): a bound item's transfer reverts `ItemLocked`, so while an item is bound `BoundTokens[item]` and the current owner cannot diverge. The asymmetry (defect 1), the cached-delegator authorization (defect 2), and the Freezer-lock divergence (defect 4) are direct code facts in the citations above; they have no dedicated runnable test.

Defect 3 is demonstrated by a runnable test that passes against the delivered runtime's real `pallet_nfts` wiring, `poc_l28_bind_adopts_and_unbind_clears_a_foreign_lock` ([pallets/highway-nft-permission/src/tests.rs#L301-L331](#)). It manufactures the precondition by calling `disable_transfer` directly (no production caller does this today), then shows `bind` swallows `ItemLocked` and records `BoundTokens` anyway, and `unbind` clears the foreign lock:

```
#[test]
fn poc_l28_bind_adopts_and_unbind_clears_a_foreign_lock() {
    new_test_ext().execute_with(|| {
        let owner = account(1);
        let buyer = account(2);
        let item =
            HighwayNftPermission::mint(&owner, &"permission".into(), &42).expect("could mint nft");
        let collection = HighwayNftPermission::collection_id().expect("collection initialised");

        // (1) Foreign lock placed directly via pallet_nfts; bind never ran, so there is
        //      no BoundTokens record backing this lock.
        assert_ok!(Nfts as Transfer<_>::disable_transfer(&collection, &item));

        // (2) bind swallows the ItemLocked error and records the binding regardless.
        assert_ok!(HighwayNftPermission::bind(&owner, &item));
        // Proof the record was written despite bind never placing the lock:
        assert_noop!(
            HighwayNftPermission::bind(&owner, &item),
            Error::::AlreadyBound
        );

        // (3) unbind clears the foreign lock: the item becomes transferable again even
        //      though this pallet never placed the lock it just removed.
        assert_ok!(HighwayNftPermission::unbind(&owner, &item));
        assert_ok!(Nfts::transfer(
            RawOrigin::Signed(owner).into(),
            collection,
            item,
            buyer,
        ));
    });
}
```

Run with `cargo test -p pallet-highway-nft-permission (test tests::poc_l28_bind_adopts_and_unbind_clears_a_foreign_lock ... ok)`.

Recommended Mitigation: One structural change closes most of the cluster. On `delegate`, escrow the NFT: transfer it to the pallet's derived account rather than only soft-locking it, and return it to `delegation.delegator` on `undelegate` / `cancel_undelegation` / `clear_relayer_delegations`. Ownership cannot drift while the pallet holds custody, so the cached-owner and current-owner can never diverge (closes defects 1 and 2 at the root).

If the soft-lock model is kept, apply the point fixes:

- Make `unbind` symmetric with `bind`: assert the caller is the current `pallet_nfts` owner, not just the `BoundTokens` record (defect 1), and have the delegation registry re-check the current owner at `undelegate` / `cancel_undelegation` / `complete_undelegation` instead of trusting the cached delegator (defect 2).
- Do not swallow `ItemLocked` in `bind`: propagate it, or fail with a typed error such as `AlreadyLockedExternally`, so the invariant "`BoundTokens[item] = X` if and only if this pallet placed the lock" holds (defect 3).

- Make `bind` aware of the other `pallet_nfts` lock conditions (the item-level `Transferable` setting and the `Locker` hook); on `unbind`, read back effective transferability and surface a distinct state or event when the NFT remains locked by another flag, instead of implying it is freely transferable (defect 4). Operationally, keep the `Freezer` role on `Highway`'s permission collection confined to the pallet's own sovereign account so no external party can introduce a competing lock.

Highway: Partially fixed in [7ea8c2b](#).

Cyfrin: Verified.

7.3.31 `clear_relayer_delegations` declares a fixed max-size proof above the runtime limits, making the normal admin path inadmissible

Description: `clear_relayer_delegations` is documented as the admin recovery operation intended to force-unbind every NFT, cancel pending exits, clear weight and tier, and make a relayer ID safe to reuse (`pallets/highway-nft-delegation-registry/src/lib.rs:633-684`). In the shipped runtime the call cannot be submitted through the normal weight-accounted admin path: `frame_system::CheckWeight` rejects the direct signed call before its body runs, even for a relayer with zero or one delegation. Wrappers that inherit the inner call's declared weight fail for the same reason. An exceptional unchecked-Sudo route can bypass this validation, as discussed under [Impact](#).

Every signed transaction runs `frame_system::CheckWeight` as a transaction extension (`runtime/src/lib.rs:76-91`). During validation, `CheckWeight::do_validate` compares the call's declared total weight with the maximum weight of one extrinsic in its dispatch class. It deliberately does not inspect current block consumption at this stage. Because this Normal call exceeds `Normal.max_extrinsic`, validation returns `InvalidTransaction::ExhaustsResources` before any of the call body executes. The relevant dimension is `proof_size`, the runtime's conservative declared PoV-weight estimate for the storage proof. The runtime also derives its whole-block proof budget from `MAX_POV_SIZE` (`runtime/src/lib.rs:217-222`).

The declared weight of this call is a constant that ignores the actual delegation count. The dispatch annotation always evaluates the generated weight at the production maximum (`pallets/highway-nft-delegation-registry/src/lib.rs:649-652`):

```
#[pallet::weight(T::WeightInfo::clear_relayer_delegations(
    T::MaxDelegationsPerRelayer::get()
))]
pub fn clear_relayer_delegations(/* ... */) -> DispatchResult {
```

`MaxDelegationsPerRelayer = 1000` (`runtime/src/configs/mod.rs:534`), and the generated proof-size formula for this call is `7_487 + 5_908 * n` bytes (`pallets/highway-nft-delegation-registry/src/weights.rs:190-202`). At the hardcoded `n = 1000`:

```
declared proof = 7_487 + 5_908 * 1000 = 5_915_487 bytes
```

The runtime block limits are:

call declared proof	= 5_915_487 bytes	(proof_size at n = 1000)
Normal max extrinsic	= 3_670_016 bytes	
Normal max total	= 3_932_160 bytes	
Operational max extrinsic	= 4_980_736 bytes	
whole block (POV)	= 5_242_880 bytes	

The declared proof is 2,245,471 bytes over the Normal per-extrinsic limit and 672,607 bytes over even the whole-block proof budget. `do_validate` rejects on the former comparison. The whole-block comparison independently shows that merely changing the call from Normal to Operational cannot make the current declaration fit. Because the annotation depends only on `MaxDelegationsPerRelayer`, and not on the stored list length, an empty or single-entry `clear` is rejected identically to a full one. No post-dispatch refund can help because validation fails before dispatch.

Files:

- `NftDelegationRegistry::clear_relayer_delegations`
- `pallet_highway_nft_delegation_registry::WeightInfo::clear_relayer_delegations`
- `frame_system::CheckWeight` (applied through the runtime's `TxExtension`)

Impact: This removes the ordinary signed forced-cleanup path. Because the documented admin recovery call is inadmissible as a normal signed extrinsic, delegation and backing state can remain attached to a relayer when an NFT holder is unavailable or unwilling to exit. The Highway admin cannot safely recycle the relayer ID or repair stale backing state through the intended call.

The severity is Low:

- Ordinary NFT holders can still exit their own delegations through the normal path; only the admin forced-cleanup path is blocked.
- There is no direct theft of funds; the defect denies a recovery capability and strands state.
- The runtime retains an exceptional `Sudo` route that can deliberately bypass the declared weight, and the supplied local/development presets assign both `Sudo` and `Highway-admin` authority to Alice. This is not the ordinary recovery path, but it materially limits the impact.

The bypass combines `sudo_unchecked_weight` with `Utility::dispatch_as(Signed(admin), clear)`. It requires `Sudo` authority and deliberately overrides the runtime resource meter. The generated 5,915,487-byte value is a conservative declared proof estimate, not a measurement proving that every full-list execution necessarily produces an encoded PoV of that size; an operator using the unchecked route must supply and justify a safe weight witness. Ordinary weight-inheriting wrappers remain inadmissible, including `Utility::batch_all([clear])` and `Sudo::sudo_as(admin, clear)`.

Proof of Concept: Add the following regression test to `runtime/src/lib.rs`. It constructs the real `RuntimeCall` and invokes the exact `CheckWeight` component used by the runtime's `TxExtension`. The test passes only when validation returns `InvalidTransaction::ExhaustsResources`, proving that the normal call is inadmissible even though the target relayer holds no delegations.

```
#[cfg(test)]
mod clear_weight_regression {
    use super::*;
    use frame_support::dispatch::GetDispatchInfo;
    use sp_runtime::transaction_validity::{InvalidTransaction, TransactionValidityError};

    #[test]
    fn empty_clear_is_rejected_before_dispatch() {
        let call = RuntimeCall::NftDelegationRegistry(
            pallet_highway_nft_delegation_registry::Call::clear_relayer_delegations {
                relayer_id: 1,
            },
        );
        let info = call.get_dispatch_info();

        assert_eq!(info.call_weight.proof_size(), 5_915_487);
        assert_eq!(
            configs::RuntimeBlockWeights::get()
                .get(frame_support::dispatch::DispatchClass::Normal)
                .max_extrinsic
                .unwrap()
                .proof_size(),
            3_670_016,
        );

        sp_io::TestExternalities::default().execute_with(|| {
            let result = frame_system::CheckWeight::do_validate(&info, 1);
            assert!(matches!(
                result,
                Err(TransactionValidityError::Invalid(
```

```

        InvalidTransaction::ExhaustsResources
    ))
    ));
}
}
}

```

Run:

```

SKIP_WASM_BUILD=1 cargo test -p parachain-template-runtime empty_clear_is_rejected_before_dispatch --
↳ --nocapture

```

With the test added, it passes by observing exactly `InvalidTransaction::ExhaustsResources`. A separate control confirmed that both `Utility::batch_all([clear])` and `Sudo::sudo_as(admin, clear)` return the same validation error because they inherit the inner call weight.

Textual Step-by-Step Proof:

1. Initial state.

- The runtime applies `frame_system::CheckWeight` to every signed transaction (runtime/src/lib.rs:76-91) and caps the whole-block proof at `MAX_POV_SIZE = 5_242_880` bytes (runtime/src/lib.rs:217-222), with Normal per-extrinsic and per-block sub-limits from `RuntimeBlockWeights` (runtime/src/configs/mod.rs:81-100).
- `MaxDelegationsPerRelayer = 1000` is wired into the pallet config (runtime/src/configs/mod.rs:534,587), and the generated `WeightInfo::clear_relayer_delegations(n)` declares proof $7_487 + 5_908 * n$ bytes (pallets/highway-nft-delegation-registry/src/weights.rs:190-202).
- A relayer holds any delegation state (or none). The state contents are irrelevant to the outcome, because the declared weight ignores them.

2. Trigger.

- The Highway admin signs `clear_relayer_delegations(relayer_id)`. `SlashOrigin = EnsureHighwayAdmin` requires the signed Highway admin.
- Runtime metadata assigns the call the fixed $n = 1000$ weight, whose `proof_size` is $7_487 + 5_908 * 1000 = 5_915_487$ bytes (pallets/highway-nft-delegation-registry/src/lib.rs:649-652).

3. Pre-dispatch rejection.

- `CheckWeight::do_validate` compares the declared 5,915,487-byte proof against the Normal per-extrinsic limit of 3,670,016 bytes and returns `InvalidTransaction::ExhaustsResources`. Validation deliberately skips current-block consumption; the separate facts that the declaration also exceeds the Normal total of 3,932,160 and whole-block proof budget of 5,242,880 are not the branch that produces this validation error.
- The origin check, `RelayerDelegations::take`, the unbind loop, pending-exit cleanup, weight and tier cleanup, and the completion event (pallets/highway-nft-delegation-registry/src/lib.rs:633-684) are never reached.

4. Result.

- The documented forced-cleanup call is unusable through the normal weight-accounted path for every relayer, regardless of actual delegation count. Repeating with fewer or zero entries does not change the declared weight. `Utility::batch_all([clear])` and `Sudo::sudo_as(admin, clear)` fail identically. Recovery requires either a runtime upgrade or the exceptional unchecked-Sudo sequence described above.

Recommended mitigation:

Make cleanup paginated and resumable, with a bounded per-call item limit whose worst-case call weight plus transaction-extension weight stays component-wise below `RuntimeBlockWeights::get().get(Normal).max_extrinsic`; separately keep the encoded transaction below `RuntimeBlockLength`. Under the current formula, 619

items fit below the Normal proof ceiling before extension overhead, while 620 already exceeds it by 431 bytes; therefore the page limit must be conservatively lower. Charging only the actual total list length is insufficient because a large list would remain inadmissible. Charge each page's validated item count in `WeightInfo::clear_relayer_delegations(n)` and return proportional post-dispatch weight. Marking the call Operational is also insufficient: the current declaration exceeds both the Operational per-extrinsic limit and the whole-block proof budget. Add a runtime test that constructs every Highway recovery call at its maximum declared dimensions and asserts `CheckWeight::do_validate` accepts it.

Highway: Fixed in [12dd67b](#), [6b7ff88](#).

Cyfrin: Verified.

7.3.32 `emit_message` accepts an over-width EVM token target and burns funds for an unclaimable message

Description: The Substrate entry pallet accepts `target_token_address` as generic bounded bytes, with a production maximum of 128 bytes (`pallets/highway-entry/src/lib.rs:831-840`; `runtime/src/configs/mod.rs:393-404`). `ChainInfo` stores only the chain ID, name, and active flag, so it carries no destination address format or expected width (`pallets/highway-config/src/lib.rs:121-125`). The shipped setup registers Ethereum as chain 3 and configures USDC as a Burn/Mint corridor, while requiring its cross-component constants to remain aligned with `hway-ethereum` and the relayer (`scripts/README.md:58-66,176-187`).

`emit_message` validates that the target chain and token corridor are usable and that the transfer amount is within the configured bounds (`pallets/highway-entry/src/lib.rs:852-867,1558-1615`). However, none of those checks validates whether the destination can represent the supplied target address. With a nonzero token ID and nonzero amount present, any nonempty target of up to 128 bytes satisfies the generic token-message shape checks (`pallets/highway-entry/src/lib.rs:1483-1515,1836-1913`).

The call then hashes the raw target bytes into the canonical message ID (`pallets/highway-entry/src/lib.rs:899-921`), burns or escrows the source value (`pallets/highway-entry/src/lib.rs:928-950,1977-2005`), and emits the same raw bytes in `MessageEmitted` (`pallets/highway-entry/src/lib.rs:952-962`).

The paired EVM executor cannot reproduce a message ID created with a 21-byte token target. Its token-only entry-point accepts `tokenTargetAddress` as a Solidity address and passes `abi.encodePacked(tokenTargetAddress)`—exactly 20 bytes—into the message-ID reconstruction (`ExecutorLogic.executeMessageToken`). The target is length-prefixed in the V1 preimage (`MessageId.generateMessageId`), so the 20-byte reconstruction cannot equal the source ID committed over 21 bytes. The executor therefore reverts with `InvalidMessageId` before minting or releasing tokens (`ExecutorLogic._verifyMessageId`).

This assumes the paired deployments use the same Highway network ID, as required for normal cross-chain delivery. With matching domains, the target-width mismatch alone makes the emitted message unclaimable under the current EVM executor. The holder cannot permissionlessly edit the committed target or reclaim the source debit; recovery requires a privileged upgrade, reissuance, or compensation.

Files:

- `pallets/highway-entry/src/lib.rs` (`emit_message`, `derive_transfer_kind`, `resolve_outbound_transfer`, and `generate_message_id`)
- `pallets/highway-config/src/lib.rs` (`ChainInfo`)
- `runtime/src/configs/mod.rs` (`MaxAddressLength`)

Impact: A holder or integrator that supplies an over-width target for an EVM-bound transfer can receive a successful source transaction even though the destination can never claim that message under the current executor. On a Burn/Mint corridor, the source supply decreases without a corresponding destination mint. On an Escrow/Release corridor, the holder's value remains locked until privileged recovery.

This is Low because the malformed target is supplied by the affected caller or its integrator, no attacker can redirect another user's funds, and the issue does not create an attacker profit. Nevertheless, the public call admits

the target as valid, commits the value movement, and provides no permissionless recovery instead of rejecting the incompatible address before the debit.

Proof of Concept: The following passing test demonstrates that a 21-byte EVM target is accepted, 100 units are burned, the nonce advances, and the exact incompatible target is committed to the event. Add it to the existing `emit_message` test module in `pallets/highway-entry/src/tests.rs`:

```
#[test]
fn evm_unrepresentable_target_is_committed_after_burn() {
    new_test_ext().execute_with(|| {
        register_test_chain(CHAIN_ID_ETHEREUM, CHAIN_NAME_ETHEREUM, true);
        register_test_token(TOKEN_ID_USDC, b"USDC");
        assert_ok!(HighwayConfig::configure_token_bridge(
            RuntimeOrigin::signed(ADMIN1),
            TOKEN_ID_USDC,
            CHAIN_ID_ETHEREUM,
            create_bridge_config_burn_mint(),
        ));

        // Keep issuance authority separate from the holder.
        assert_ok!(Assets::create(
            RuntimeOrigin::signed(ADMIN1),
            TOKEN_ID_USDC,
            ADMIN1,
            1,
        ));
        assert_ok!(Assets::mint(
            RuntimeOrigin::signed(ADMIN1),
            TOKEN_ID_USDC,
            1,
            1_000,
        ));

        // The paired EVM executor can reconstruct only a 20-byte address.
        let raw_21_byte_target = vec![0x11u8; 21];
        assert_ok!(HighwayEntry::emit_message(
            RuntimeOrigin::signed(1),
            CHAIN_ID_ETHEREUM,
            None,
            None,
            Some(BoundedVec::try_from(raw_21_byte_target.clone()).unwrap()),
            Some(TOKEN_ID_USDC),
            Some(100),
            None,
        ));

        // The incompatible message was accepted and its source value was burned.
        assert_eq!(Assets::balance(TOKEN_ID_USDC, 1), 900);
        assert_eq!(Assets::total_issuance(TOKEN_ID_USDC), 900);
        assert_eq!(MessageNonce:::<Test>::get(CHAIN_ID_ETHEREUM), 1);

        let (emitted_target, emitted_nonce) = System::events()
            .iter()
            .find_map(|record| match &record.event {
                RuntimeEvent::HighwayEntry(Event::MessageEmitted {
                    target_chain_id,
                    target_token_address,
                    nonce,
                    ..
                }) if *target_chain_id == CHAIN_ID_ETHEREUM => {
                    Some((target_token_address.clone(), *nonce))
                }
            })
            .=> None,
```

```

    })
    .expect("MessageEmitted must be deposited");
    assert_eq!(
        emitted_target.unwrap().as_slice(),
        raw_21_byte_target.as_slice()
    );
    assert_eq!(emitted_nonce, 1);
  });
}

```

Run:

```
cargo test -p pallet-highway-entry evm_unrepresentable_target_is_committed_after_burn -- --nocapture
```

Result:

```

running 1 test
test ... evm_unrepresentable_target_is_committed_after_burn ... ok

test result: ok. 1 passed; 0 failed

```

The passing assertions show the vulnerable terminal state: the holder and total issuance are both reduced to 900, nonce 1 is consumed, and the event preserves all 21 target bytes. No call to the paired `executeMessageToToken(address, ...)` ABI can supply those same bytes, so its reconstructed ID necessarily differs.

Textual Step-by-Step Proof:

1. Ethereum is registered as an active target and a normal Burn/Mint asset corridor is configured. The paired deployments use matching Highway network IDs.
2. A holder with 1,000 units calls `emit_message` with a 21-byte `target_token_address` and amount 100.
3. The target is below `MaxAddressLength = 128`; the chain, corridor, token, transfer shape, and amount checks pass because none enforces the EVM address width.
4. Substrate generates the message ID over all 21 target bytes, burns 100 units, advances the nonce to 1, and emits the same target bytes.
5. The EVM token-only executor can accept only a 20-byte address. It therefore reconstructs a different V1 preimage and reverts with `InvalidMessageId` before destination value movement.
6. The holder cannot alter the authenticated source fields or reverse the source debit permissionlessly. The message remains unclaimable under the current executor absent privileged recovery.

Recommended mitigation: Store the expected address length in each chain's configuration and reject every nonempty target with a mismatched length before nonce mutation, fee collection, or value movement. Configure Ethereum as 20 bytes and add 19-, 20-, and 21-byte tests.

Highway: Acknowledged. Destination width is treated as a caller/SDK responsibility; malformed raw calls affect only the caller's funds, with privileged administrative compensation accepted as recovery.

Cyfrin: Rationale accepted: a malformed destination width can strand the signing account's own value, but the call cannot debit another account or create attacker profit, and Highway expressly accepts that recovery requires privileged reissue or compensation.

7.4 Informational

7.4.1 `dev_chain_spec.json` genesis is stale versus the audited runtime and encodes a plural `highwayConfig.admins` array

Description: The checked-in raw chain spec `dev_chain_spec.json` (referenced as the launch spec by `zombienet-omni-node.toml`) carries a `highwayConfig` genesis patch of `{ "admins": [Alice, Bob] }` - a plural `admins` array (`dev_chain_spec.json:68-73`). The audited runtime's `highway-config` pallet has no such field: its `admin` state is a single `Admin StorageValue` seeded from a singular genesis `admin` field (`pallets/highway-config/src/lib.rs:222-224`), and the genesis build path writes that one value (`BuildGenesisConfig::build`, `pallets/highway-config/src/lib.rs:415`). The spec's `highwayConfig` patch also omits the `network_id` value the current genesis config requires. The plural `admins` key does not exist in the runtime's genesis schema and the required `network_id` is absent, so the spec no longer matches the runtime it is meant to instantiate: it was generated against an older, divergent runtime whose `admin` model differed from the single-`Admin` model under audit.

Files:

- `BuildGenesisConfig::build`

Impact: Bootstrapping from this stale spec deploys a misconfigured genesis: the launch artifact and the audited runtime disagree on the `highway-config` genesis schema (a plural two-admin array versus a single `Admin` value, plus a missing required `network_id`), so the documented omni-node launch path does not stand up the runtime that was reviewed. This is a deployment-artifact correctness defect, not an on-chain exploit, and it is recoverable by regenerating the spec from the current runtime.

Recommended Mitigation: Regenerate `dev_chain_spec.json` from the current audited runtime so its genesis patch matches the runtime schema - a singular `admin` field plus the required `network_id` - instead of the stale plural `admins` array. Add a CI step that rebuilds the checked-in spec from the runtime presets and diffs it against the committed file, so genesis-schema drift between the deployment artifact and the runtime fails the build rather than shipping silently.

Highway: Fixed in [4dd2f5c](#).

Cyfrin: Verified.